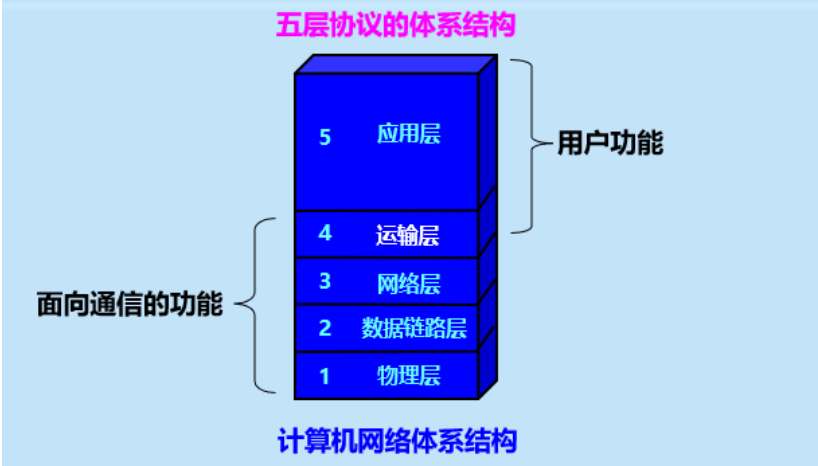
5.1、运输层概述

概念

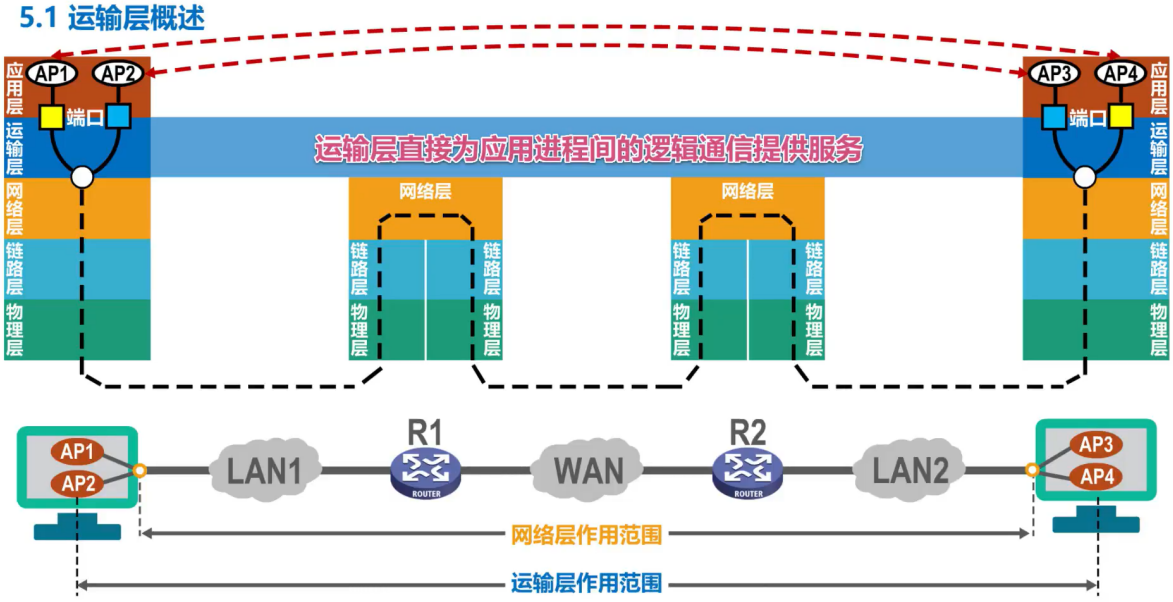
进程之间的通信



- 从通信和信息处理的角度看，运输层向它上面的应用层提供通信服务，**它属于面向通信部分的最高层，同时也是用户功能中的最低层。**
- 当网络的边缘部分中的两个主机使用网络的核心部分的功能进行端到端的通信时，**只有位于网络边缘部分的主机的协议栈才有运输层**，而网络核心部分中的路由器在转发分组时都只用到三层（到网络层）的功能。

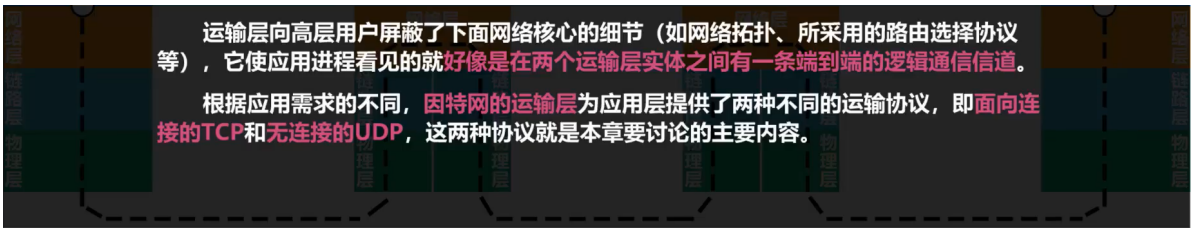
- 之前课程所介绍的计算机网络体系结构中的物理层、数据链路层以及网络层它们共同解决了将主机通过异构网络互联起来所面临的问题，**实现了主机到主机的通信。**
- 但实际上在计算机网络中进行**通信的真正实体是位于通信两端主机中的进程。**
- **如何为运行在不同主机上的应用进程提供直接的通信服务是运输层的任务**，运输层协议又称为端到端协议。

进程之间通信流程



“逻辑通信”是指运输层之间的通信好像是沿水平方向传送数据，但事实上，这两条数据并没有一条水平方向的物理连接，要传送的数据是沿着图中上下多次的虚线方向传送的

> 进程Ap1与Ap4之间进行基于网络的通信，进程Ap2与Ap3之间进行基于网络的通信
>
> 在运输层使用不同的端口，来对应不同的应用进程
>
> 然后通过网络层及其下层来传输应用层报文
>
> 接收方的运输层通过不同的端口，将收到的应用层报文，交付给应用层中相应的应用进程
>
> 这里端口并不是指看得见、摸得着的物理端口，而是指用来区分不同应用进程的标识符



总结

- 之前课程所介绍的计算机网络体系结构中的物理层、数据链路层以及网络层它们共同解决了将主机通过异构网络互联起来所面临的问题，**实现了主机到主机的通信。**
- 但实际上在计算机网络中进行**通信的真正实体是位于通信两端主机中的进程。**
- **如何为运行在不同主机上的应用进程提供直接的通信服务是运输层的任务**，运输层协议又称为端到端协议。
- 运输层向高层用户屏蔽了下面网络核心的细节（如网络拓扑、所采用的路由选择协议等），它使应用进程看见的就**好像是在两个运输层实体之间有一条端到端的逻辑通信信道。**
- 根据应用需求的不同，因特网的运输层为应用层提供了两种不同的运输协议，即**面向连接的TCP**和**无连接的UDP**，这两种协议就是本章要讨论的主要内容。

端系统之间通信的含义

“主机A和主机B进行通信”实际上是指：“运行在主机A上的某个程序和运行在主机B上的另一个程序进行通信”。端到端的通信是进程之间的通信。

即“主机 A 的某个进程和主机 B 上的另一个进程进行通信”。

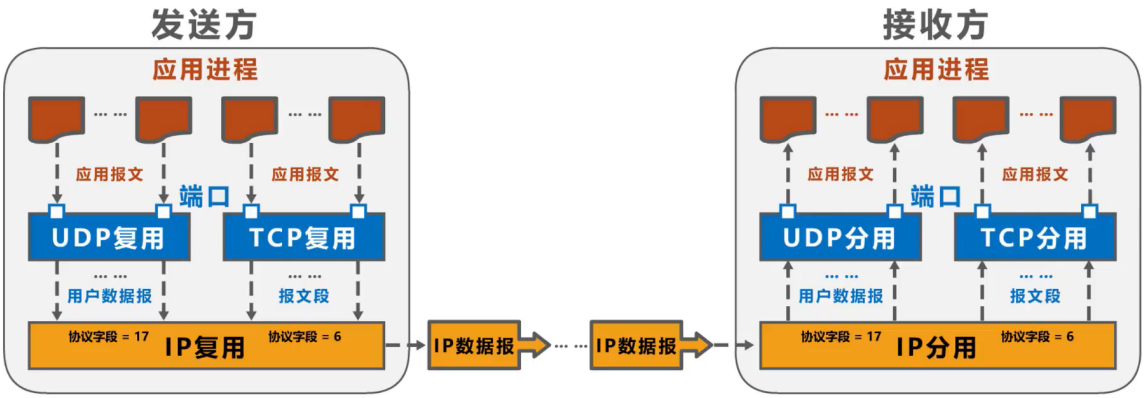
简称为“计算机之间通信”。

5.2、运输层端口号、复用与分用的概念

为什么用端口号

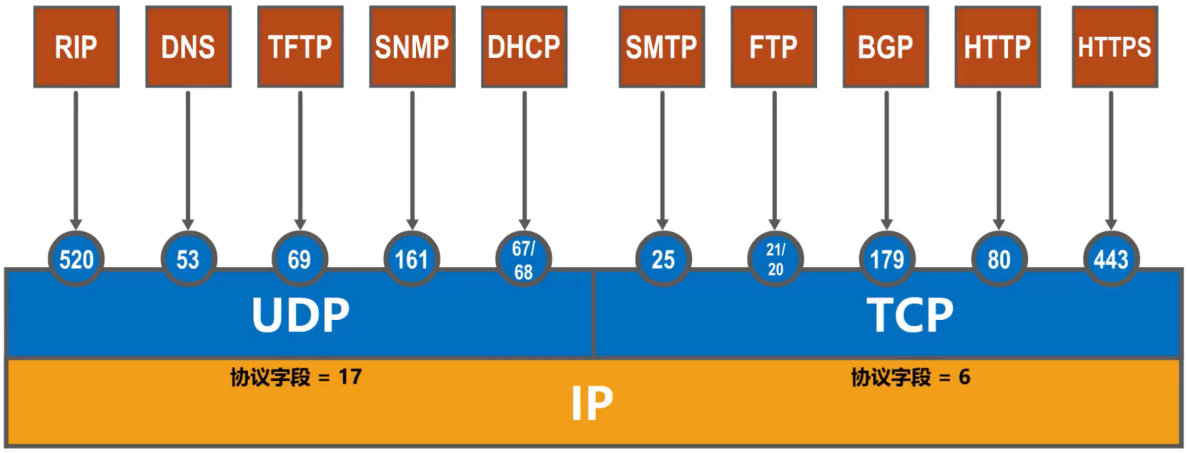
- 运行在计算机上的进程使用**进程标识符PID**来标志。
- 因特网上的计算机并不是使用统一的操作系统，不同的操作系统（windows，Linux，Mac OS）又使用**不同格式的进程标识符**。
- 为了使运行不同操作系统的计算机的应用进程之间能够进行网络通信，就必须**使用统一的方法对TCP/IP体系的应用进程进行标识**。
- TCP/IP体系的运输层使用**端口号**来区分应用层的不同应用进程。
  - 端口号使用**16比特表示**，取值范围**0~65535**;
    - **熟知端口号**：0~1023，IANA把这些端口号指派给了TCP/IP体系中最重要的一些应用协议，例如：FTP使用21/20，HTTP使用80，DNS使用53。
    - **登记端口号**：1024~49151，为没有熟知端口号的应用程序使用。使用这类端口号必须在IANA按照规定的手续登记，以防止重复。例如：Microsoft RDP 微软远程桌面使用的端口是3389。
    - **短暂端口号**：49152~65535，留给客户进程选择暂时使用。当服务器进程收到客户进程的报文时，就知道了客户进程所使用的动态端口号。通信结束后，这个端口号可供其他客户进程以后使用。
  - **端口号只具有本地意义**，即端口号只是为了**标识本计算机应用层中的各进程**，在因特网中，**不同计算机中的相同端口号是没有联系的**。

发送方的复用和接收方的分用



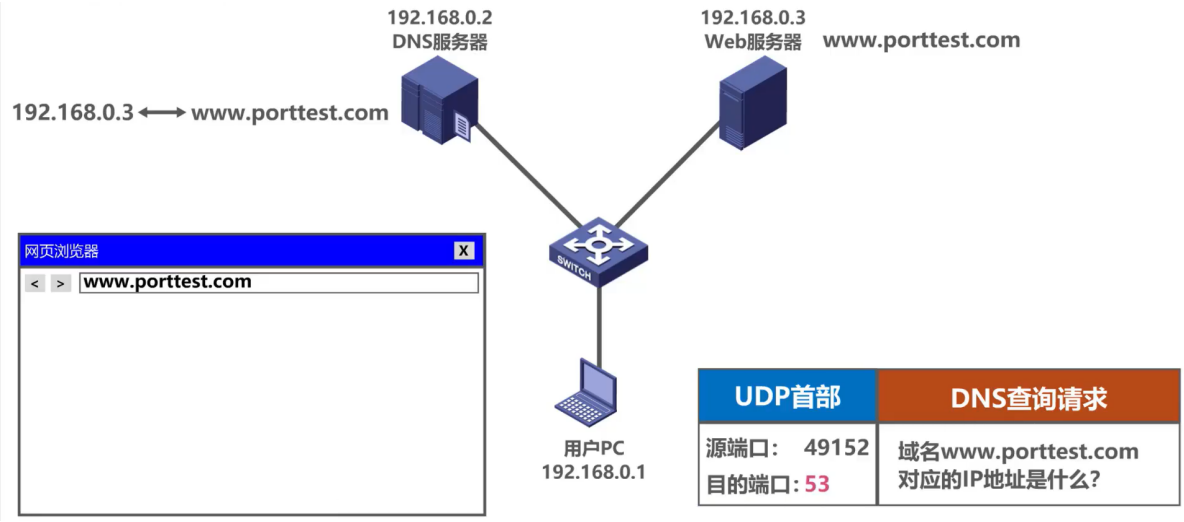
> **多个进程（这里一个端口表示一个进程）** 利用一个运输层协议（或者称为运输层接口）**发送**数据称为 **复用**
>
> **多个进程（这里一个端口表示一个进程）** 利用一个运输层协议（或者称为运输层接口）**接收**时叫做 **分用**。

TCP/IP体系的应用层常用协议所使用的运输层熟知端口号



运输层传输流程

举例



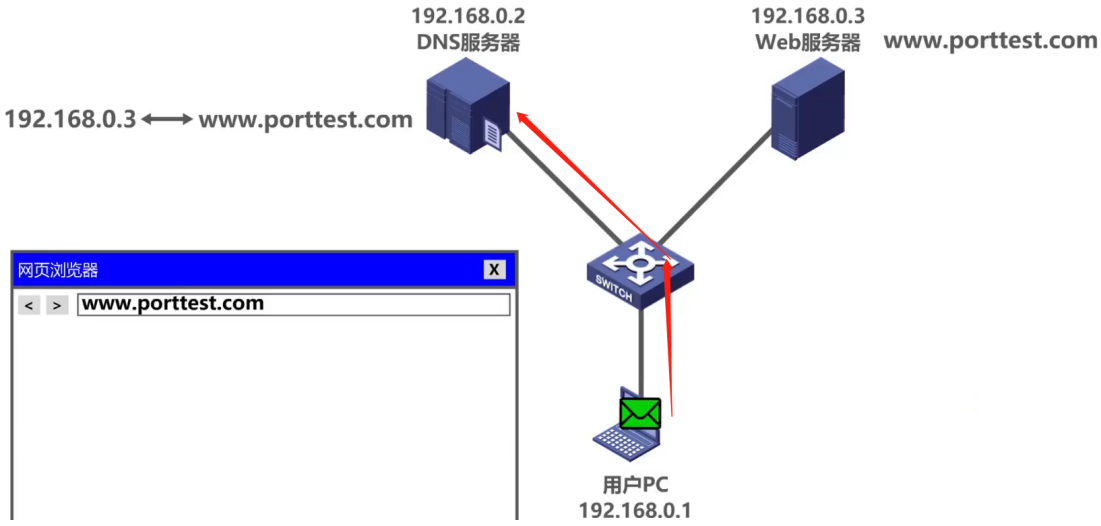
在浏览器输入域名，回车浏览

然后用户PC中的DNS客户端进程会发送一个DNS查询请求报文

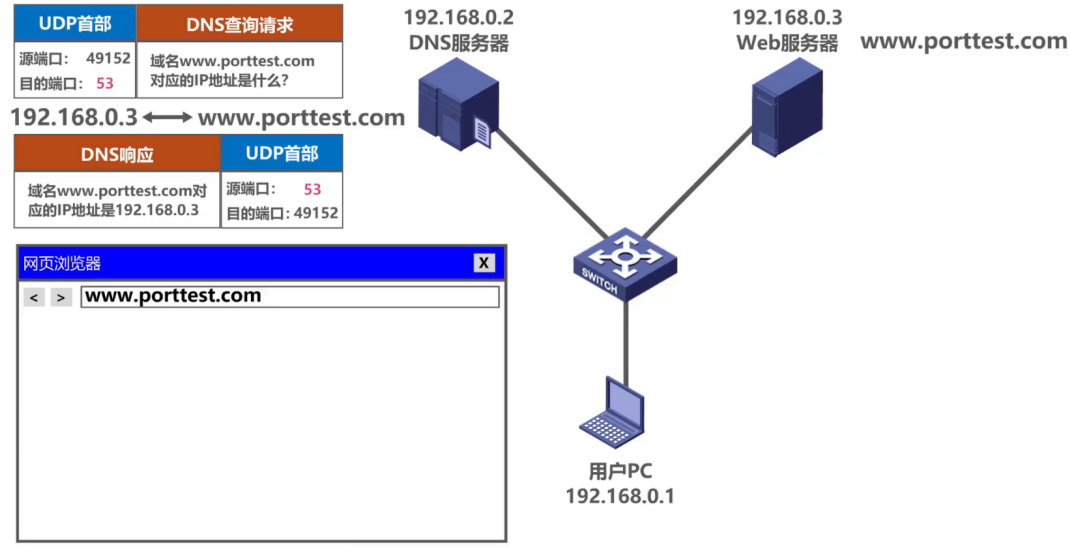
DNS查询请求报文需要使用运输层的UDP协议

首部中的源端口字段的值，在短暂端口号49151~65535中挑选一个未被占用的，用来表示DNS客户端进程

首部中的目的端口字段的值：53，是DNS服务器端进程所使用的熟知端口号



之后，将UDP用户数据报封装在IP数据报中，通过以太网发送给DNS服务器



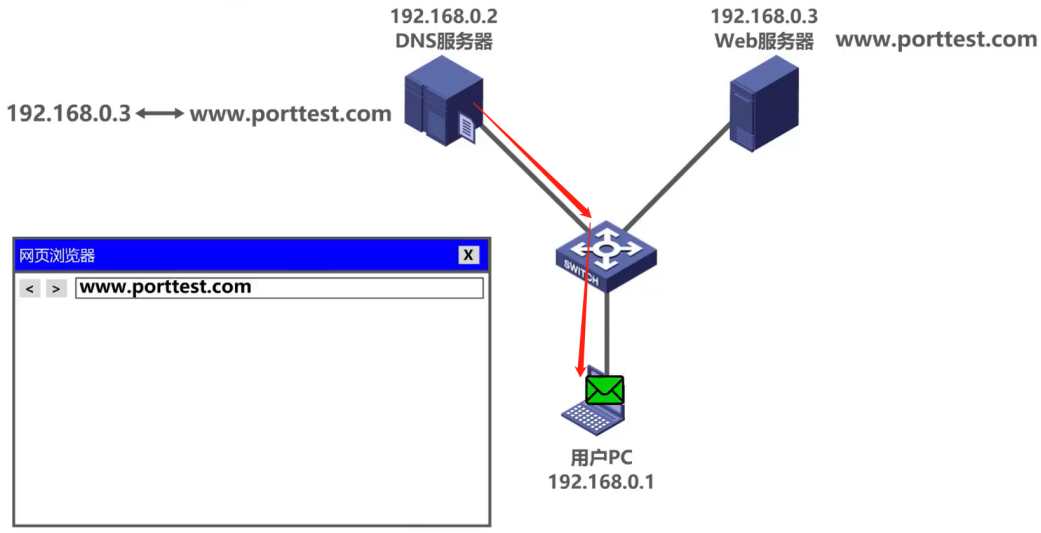
DNS服务器收到该IP数据报后，从中解封出UDP用户数据报

UDP首部中的目的端口号为53，这表明应将该UDP用户数据报的数据载荷部分，也就是DNS查询请求报文，交付给本服务器中的DNS服务器端进程

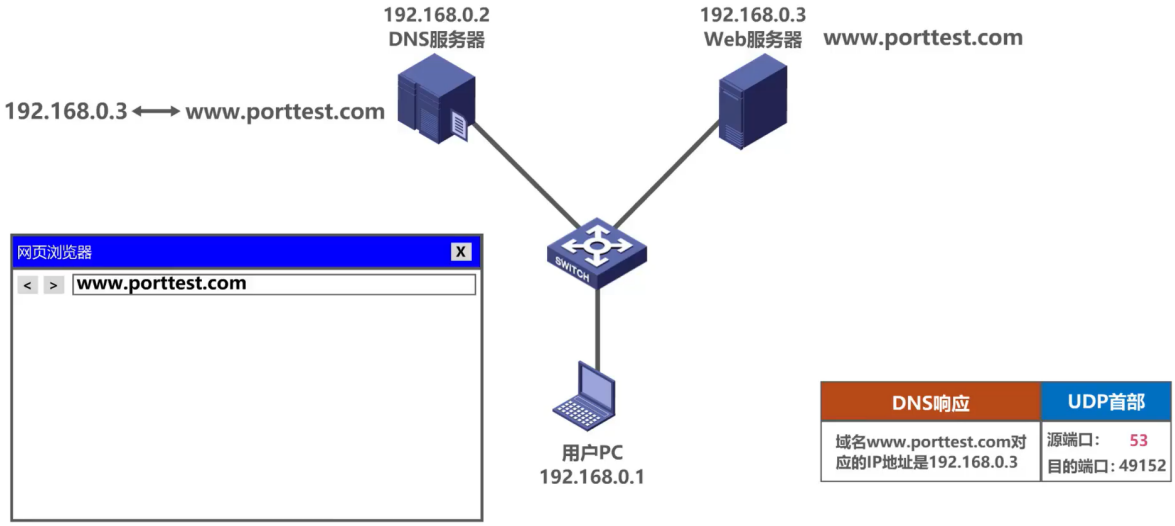
DNS服务器端进程解析DNS查询请求报文的内容，然后按其要求查找对应的IP地址

之后，会给用户PC发送DNS响应报文，DNS响应报文需要使用运输层的UDP协议封装成UDP用户数据报

其首部中的源端口字段的值设置为熟知端口号53，表明这是DNS服务器端进程所发送的UDP用户数据报，目的端口的值设置为49152，这是之前用户PC中发送DNS查询请求报文的DNS客户端进程所使用的短暂端口号

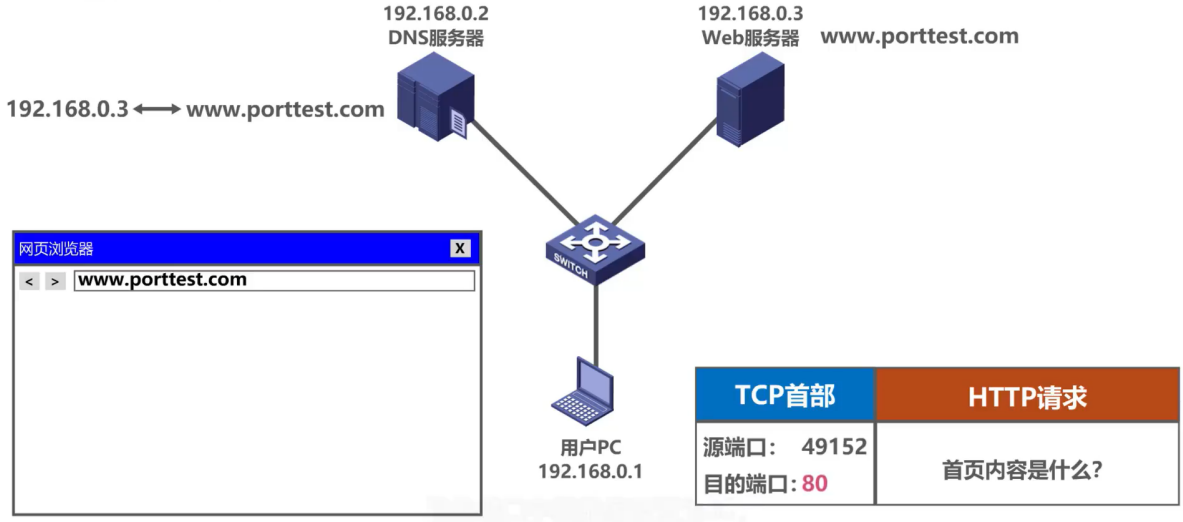


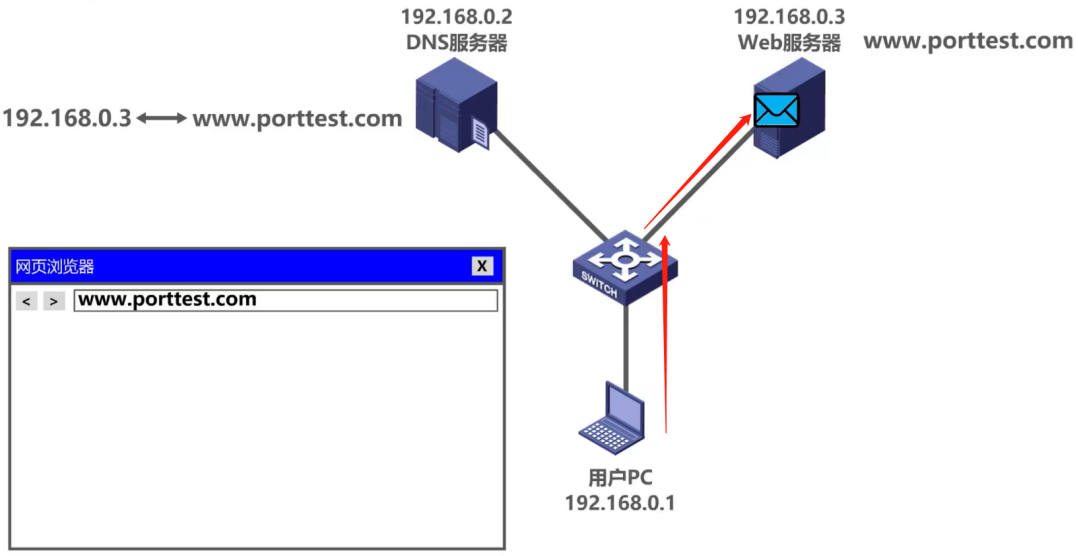
> 将UDP用户数据报封装在IP数据报中，通过以太网发送给用户PC



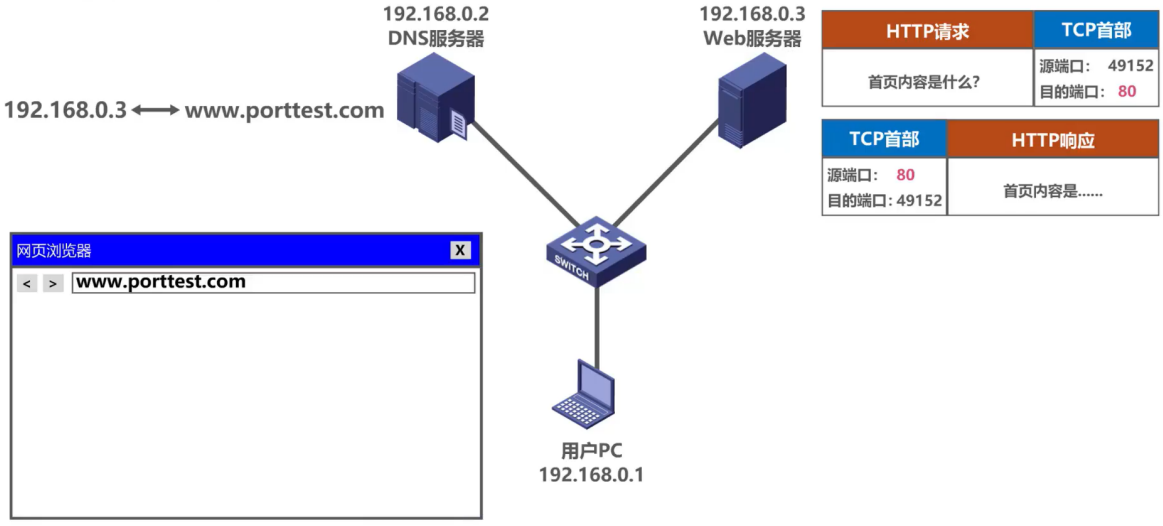
> 用户PC收到该数据报后，从中解封出UDP用户数据报
>
> UDP首部中的目的端口号为49152，这表明应将该UDP用户数据报的数据载荷部分，也就是DNS响应报文，交付给用户PC中的DNS客户端进程
>
> DNS客户端进程解析DNS响应报文的内容，就可知道自己之前所请求的Web服务器的域名对应的IP地址

现在用户PC中的HTTP客户端进程可以向Web服务器发送HTTP请求报文（和DNS发送和接收流程差不多）

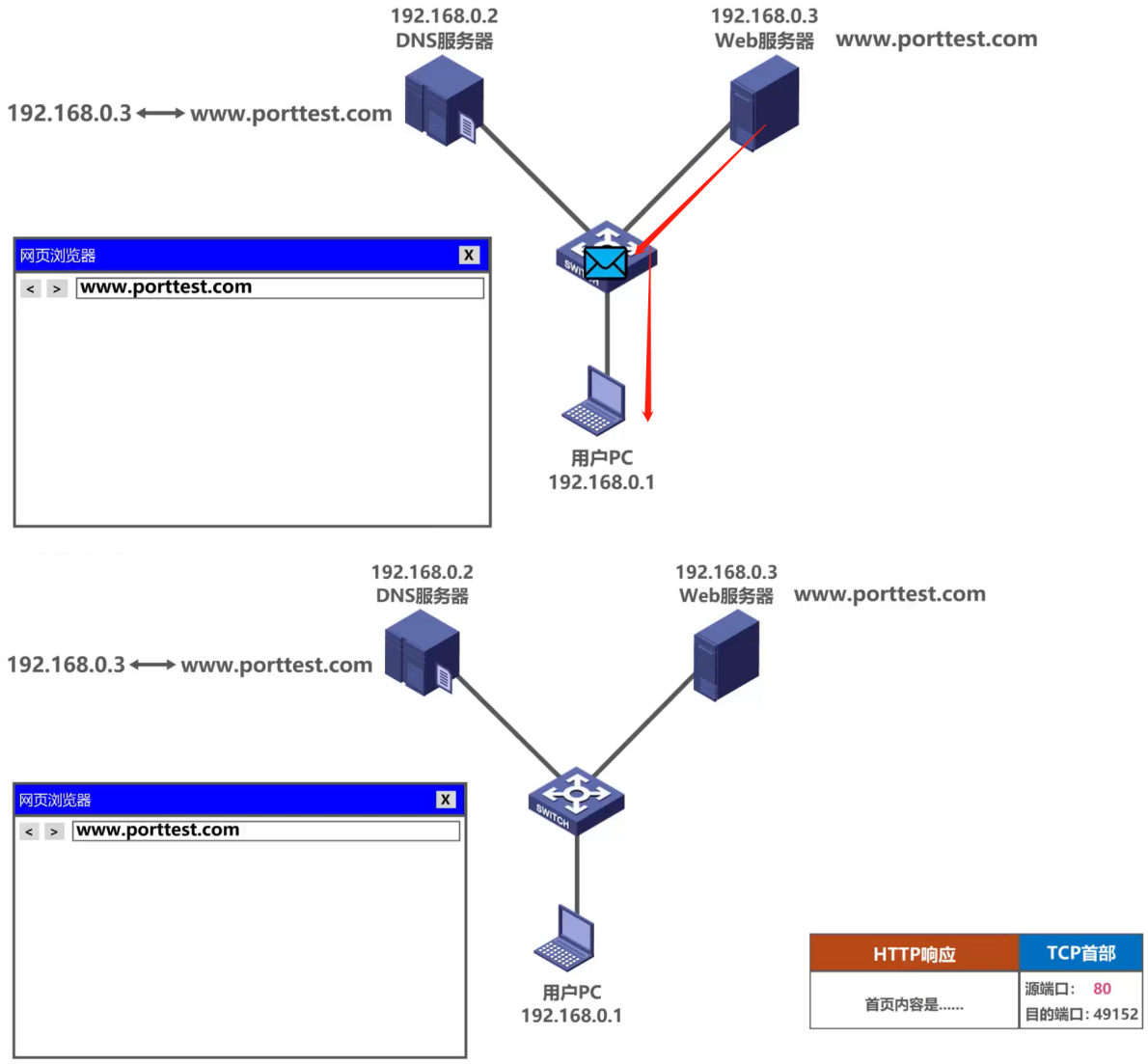

192.168.0.2
DNS服务器
192.168.0.3
Web服务器
www.porttest.com
192.168.0.3 ⟷ www.porttest.com
网页浏览器
www.porttest.com
SWITCH
用户PC
192.168.0.1
TCP首部
HTTP请求
源端口： 49152
目的端口: 80
首页内容是什么?


192.168.0.2
DNS服务器
192.168.0.3
Web服务器
www.porttest.com
192.168.0.3 ⟷ www.porttest.com
网页浏览器
www.porttest.com
SWITCH
用户PC
192.168.0.1

【举例】运输层端口号


192.168.0.2
DNS服务器
192.168.0.3
Web服务器
192.168.0.3 ⟷ www.porttest.com
网页浏览器
www.porttest.com
SWITCH
用户PC
192.168.0.1
HTTP请求
TCP首部
首页内容是什么?
源端口： 49152
目的端口： 80
TCP首部
HTTP响应
源端口： 80
目的端口: 49152
首页内容是.....

【举例】运输层端口号

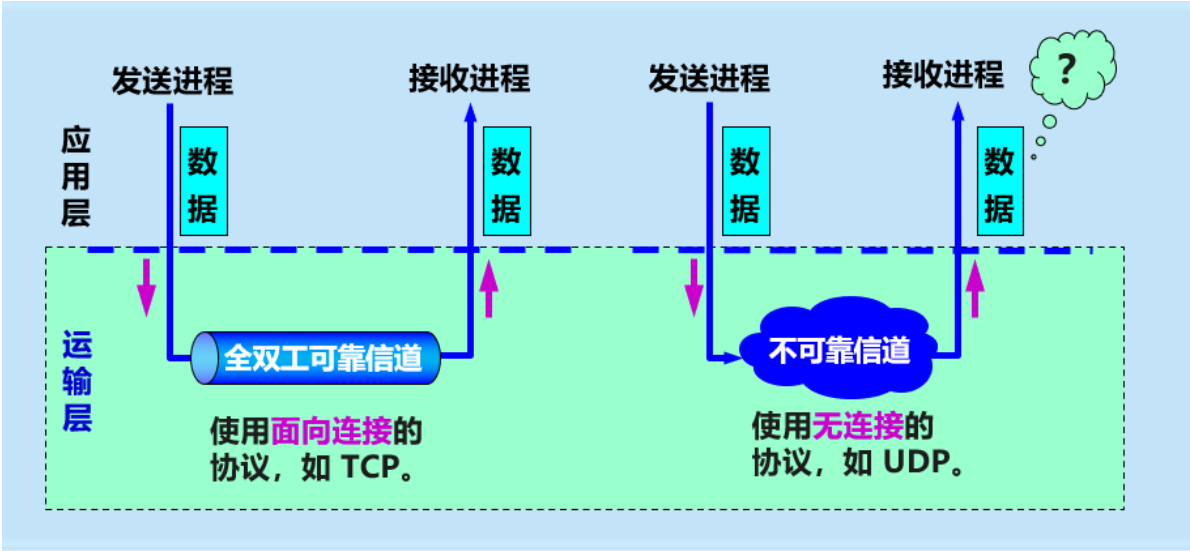


5.3、 UDP和TCP的对比

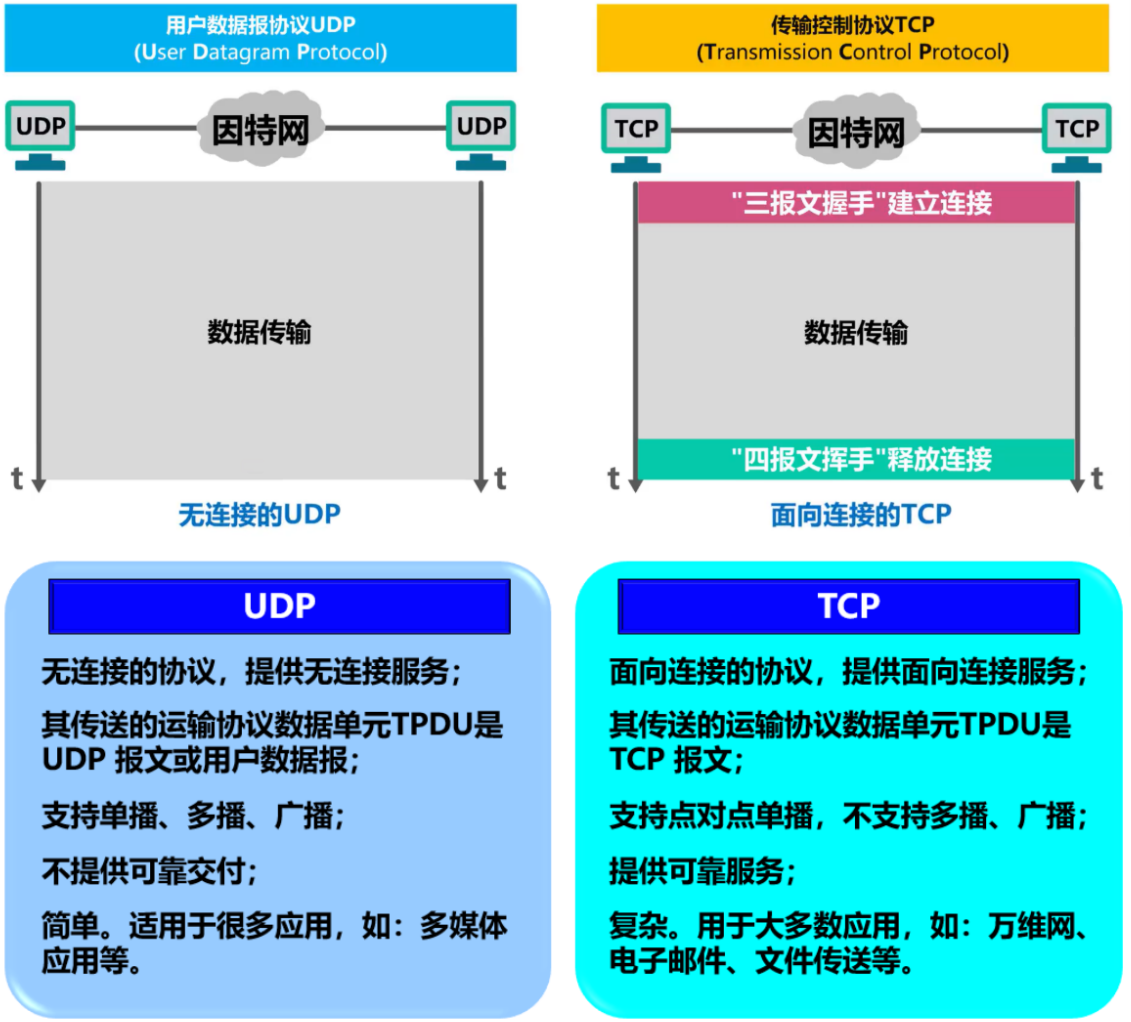
概念

- UDP 和 TCP 是TCP/IP体系结构**运输层**中的两个重要协议
- 当运输层采用面向连接的 TCP 协议时，尽管下面的网络是不可靠的（只提供尽最大努力服务），但这种逻辑通信信道就相当于一条**全双工的可靠信道**。
- 当运输层采用无连接的 UDP 协议时，这种逻辑通信信道是一条**不可靠信道**。

可靠信道与不可靠信道



- 两个对等运输实体在通信时传送的数据单位叫作**运输协议数据单元** TPDU (Transport Protocol Data Unit)。
- TCP 传送的数据单位协议是 **TCP 报文段**(segment)。
- UDP 传送的数据单位协议是 **UDP 报文**或**用户数据报**。

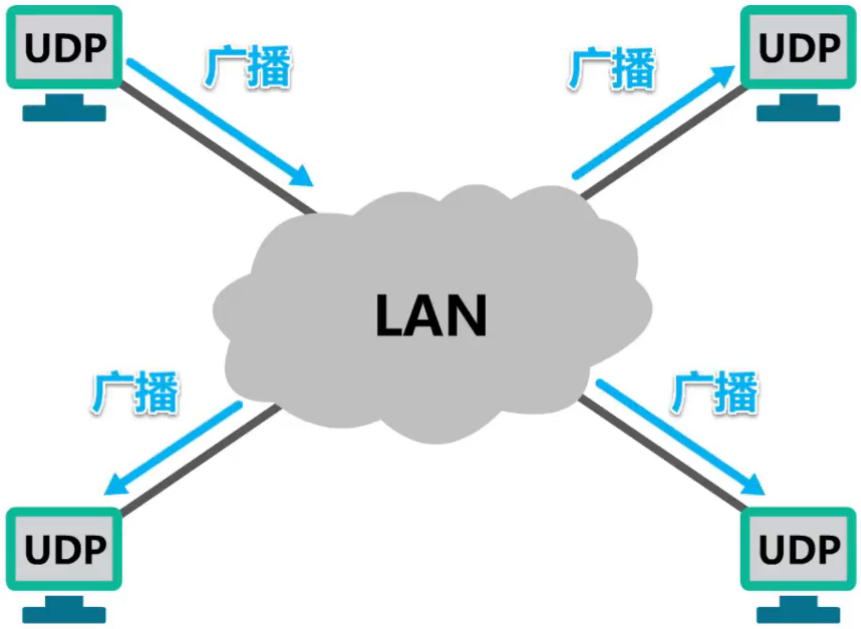


UDP的通信是无连接的，不需要套接字（Socket）

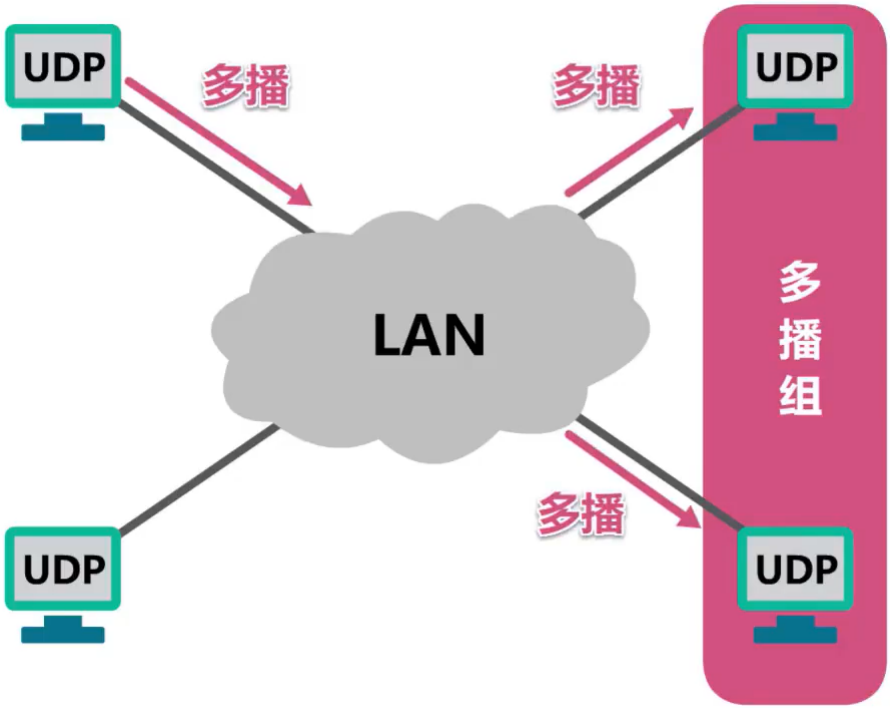
TCP是面向连接的，TCP之间的通信必须要在两个套接字（Socket）之间建立连接

用户数据报协议UDP（User Datagram Protocol）

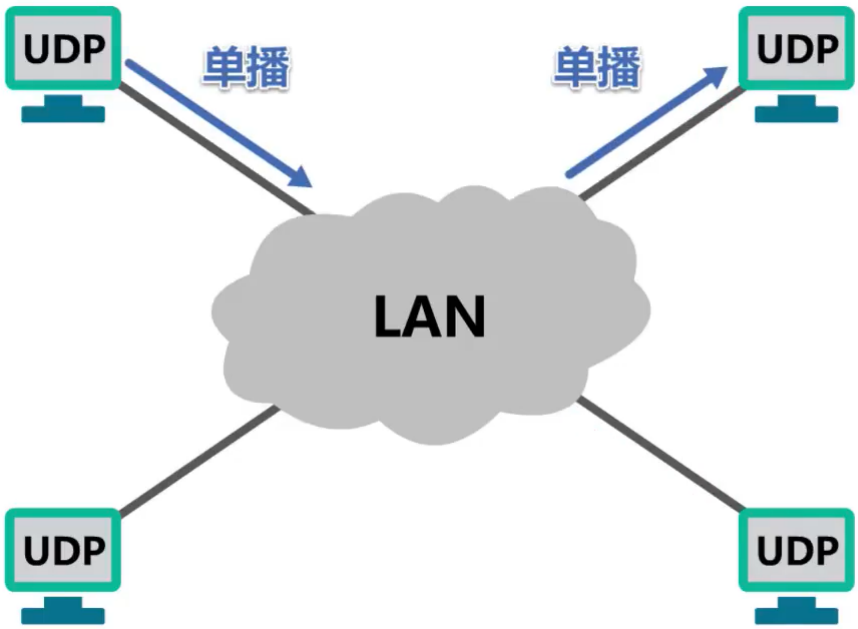
可以发送广播



可以向某个多播组发送多播

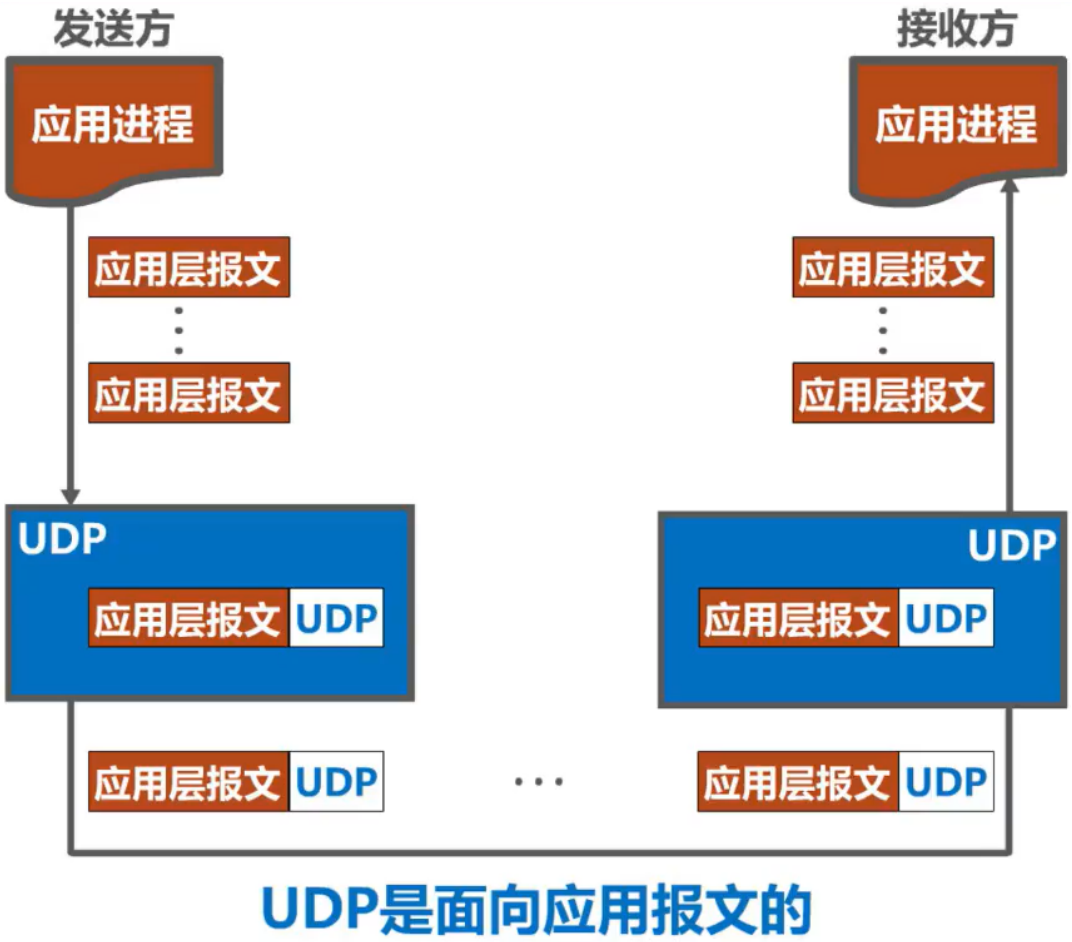


还可以发送单播



> UDP 支持单播、多播以及广播
>
> 换句话说，UDP支持一对一，一对多，以及一对全的通信

运输过程

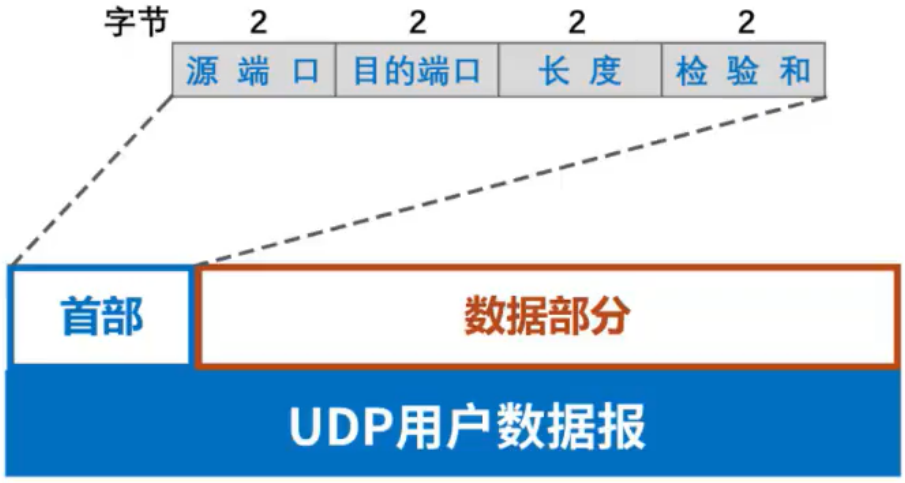


> UDP对应用进程交下来的报文既不合并也不拆分，而是保留这些报文的边界
>
> 换句话说，UDP是面向应用报文的

UDP向上层提供无连接不可靠传输服务

UDP向上层提供无连接不可靠传输服务
（适用于IP电话、视频会议等实时应用）

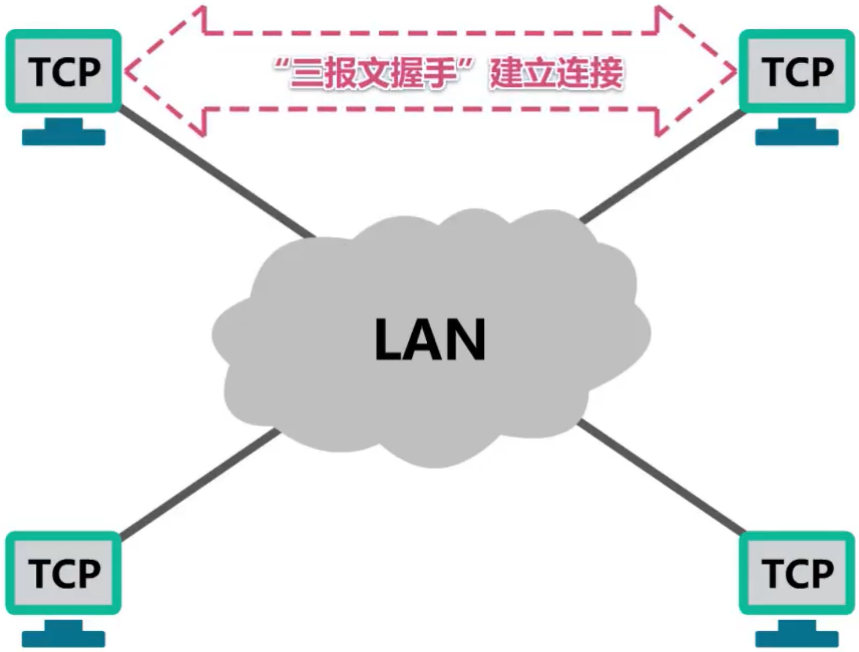
UDP结构



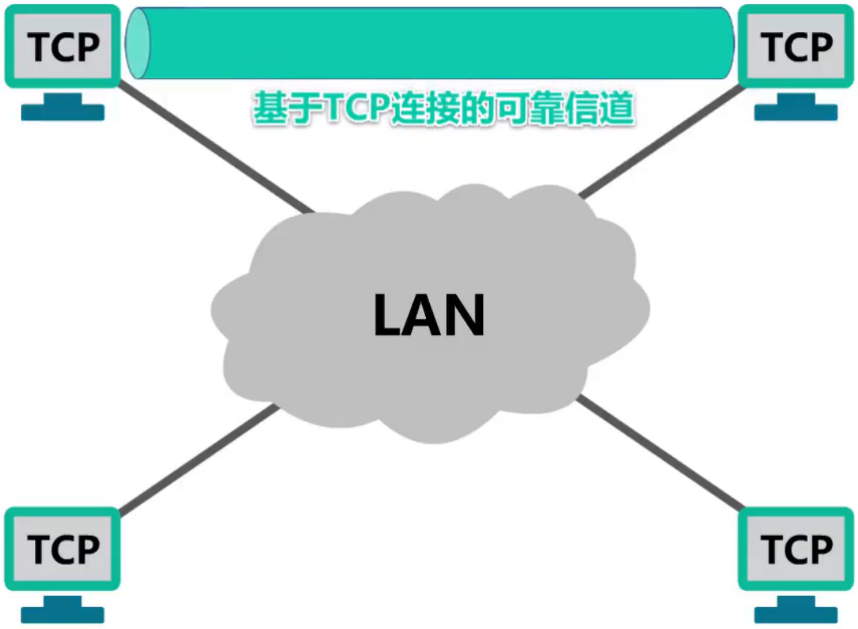
UDP用户数据报首部仅8字节

传输控制协议TCP （Transmission Control Protocol）

使用TCP协议的通信双方，在进行数据传输之前，必须使用“三报文握手”建立TCP连接

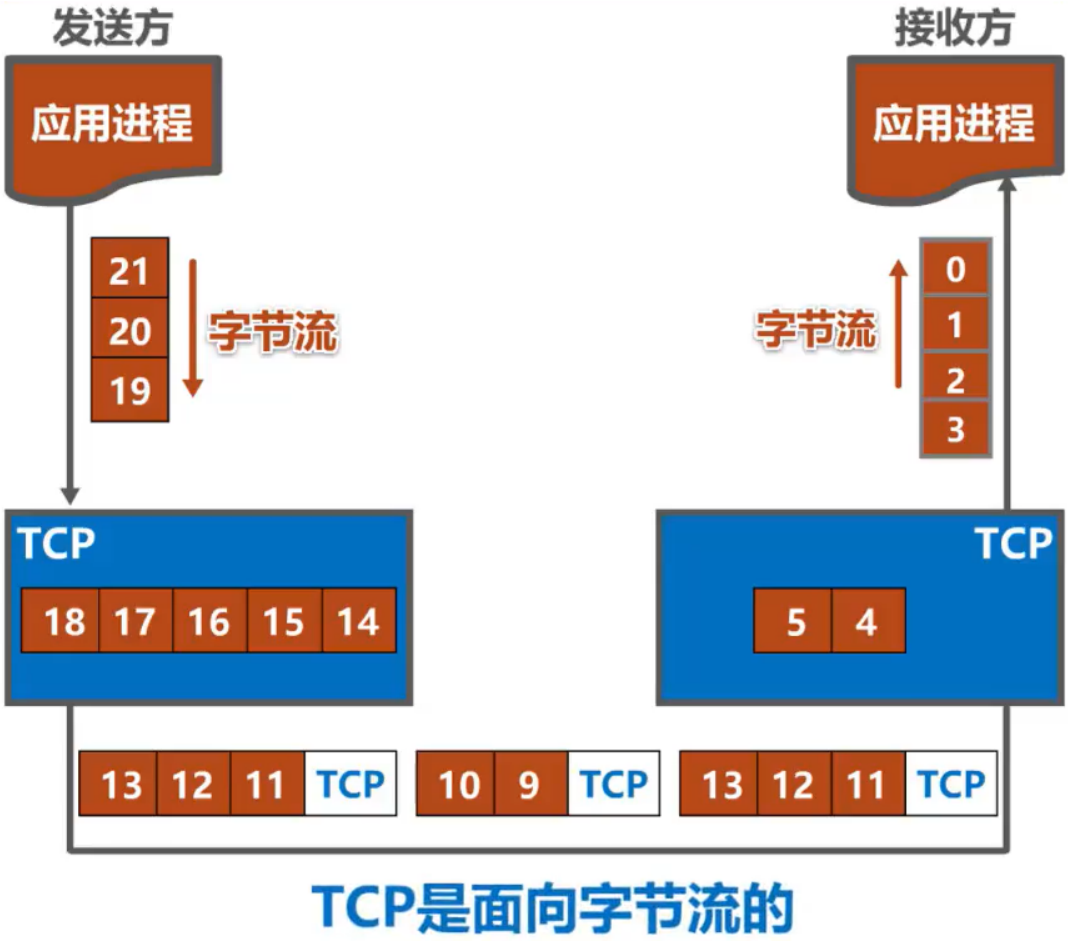


TCP连接建立成功后，通信双方之间就好像有一条可靠的通信信道，通信双方使用这条基于TCP连接的可靠信道进行通信



> 很显然，TCP仅支持单播，也就是一对一的通信

运输过程



发送方

- TCP会把应用进程交付下来的数据块看作是一连串无结构的字节流，TCP并不知道这些待传送的字节流的含义
- 并将他们编号，并存储在自己发送缓存中
- TCP会根据发送策略，提取一定量的字节构建TCP报文并发送

接收方

- 一方面从所接受到的TCP报文段中，取出数据载荷部分并存储在接收缓存中；一方面将接收缓存中的一些字节交付给应用进程
- TCP不保证接收方应用进程所收到的数据块与发送方发送的数据块，具有对应大小的关系（例如，发送方应用进程交给发送方的TCP共10个数据块，但接收方的TCP可能只用了4个数据块，就把收到的字节流交付给了上层的应用进程，但接收方收到的字节流必须和发送方应用进程发出的字节流完全一样）
- 接收方的应用进程必须有能力识别收到的字节流，把它还原成有意义的应用层数据

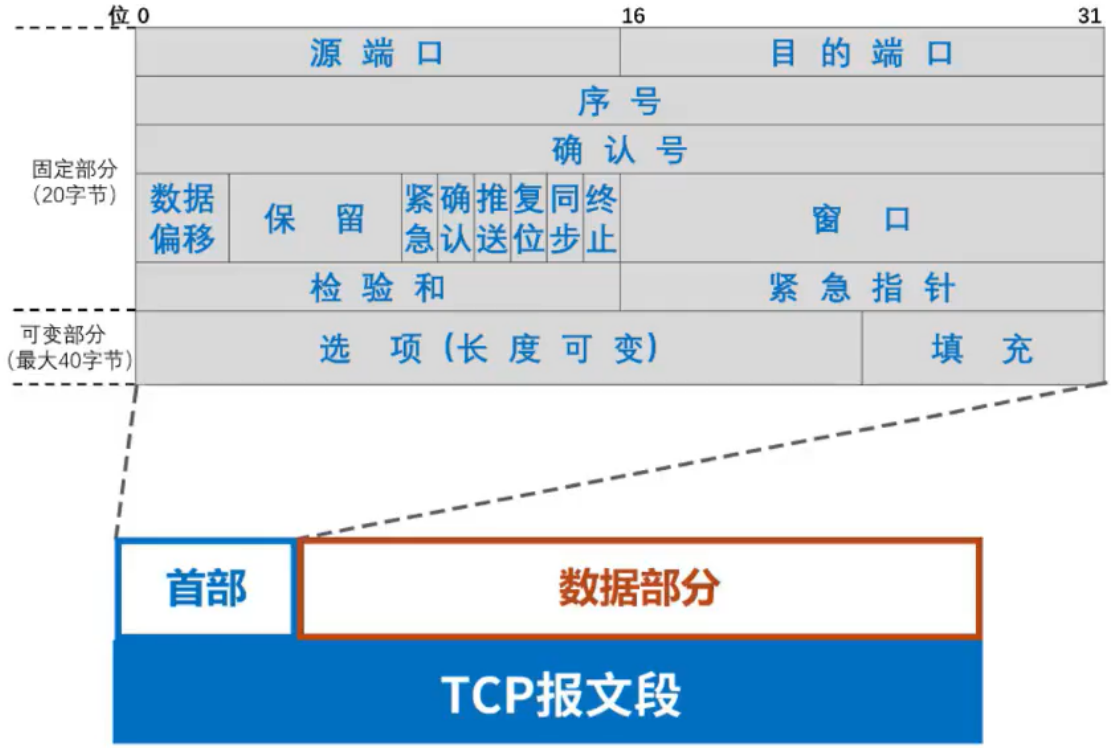
TCP是面向字节流的，这正是TCP实现可靠传输、流量控制、以及拥塞控制的基础

本图只画了一个方向的数据流，在实际网络中，基于TCP连接的两端，可以同时进行TCP报文段的发送和接收

TCP向上层提供面向连接的可靠传输服务

TCP向上层提供面向连接的可靠传输服务
（适用于要求可靠传输的应用，例如文件传输）

TCP结构



TCP报文段首部最小20字节，最大60字节

总结

UDP

- 无连接
- 支持一对一，一对多，多对一和多对多交互通信。
- 对应用层交付的报文直接打包
- 尽最大努力交付，也就是不可靠；不使用流量控制和拥塞控制。
- 首部开销小，仅8字节

TCP

- 面向连接
- 每一条TCP连接只能有两个端点EP，只能是一对一通信。
- 面向字节流
- 可靠传输，使用流量控制和拥塞控制。
- 首部最小20字节，最大60字节

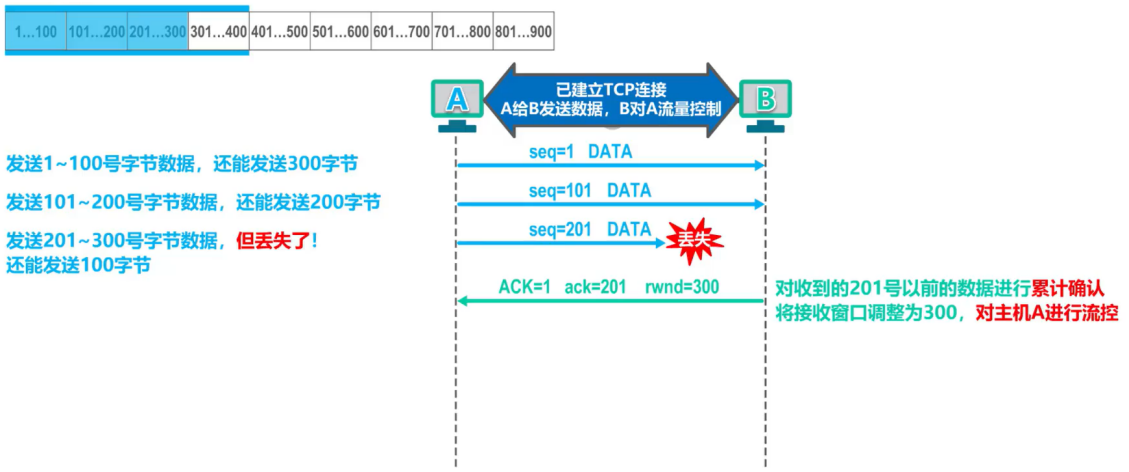
5.4、TCP的流量控制

概念

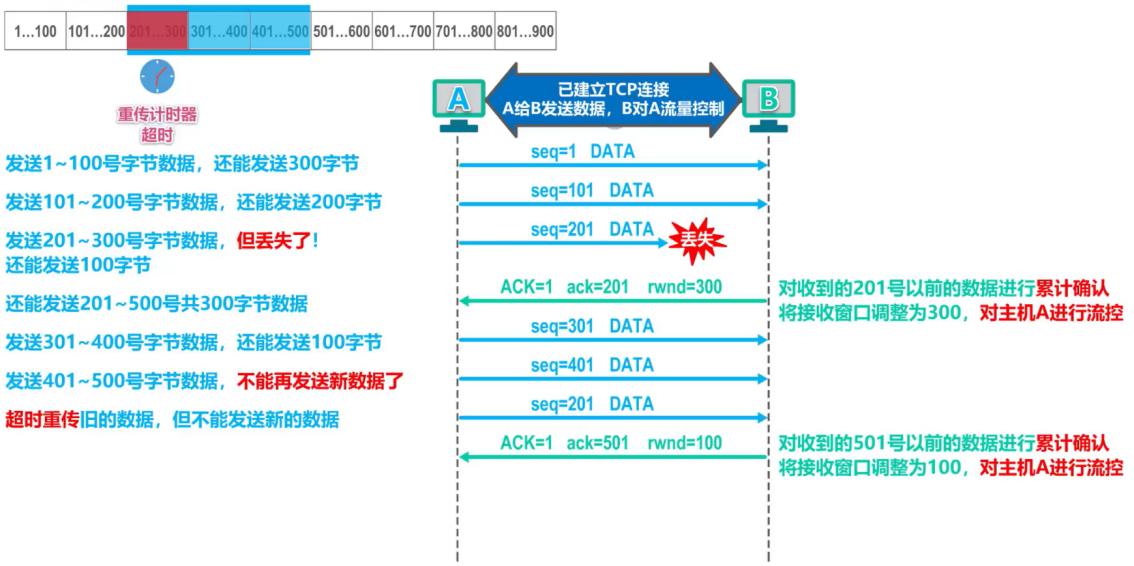
- 一般来说，我们总是希望数据传输得更快一些。
  - 但如果发送方把数据发送得过快，接收方就可能来不及接收，这就会造成数据的丢失。
- 所谓流量控制（flow control）就是**让发送方的发送速率不要太快，要让接收方来得及接收**。
- 利用**滑动窗口**机制可以很方便地在TCP连接上实现对发送方的流量控制。

举例

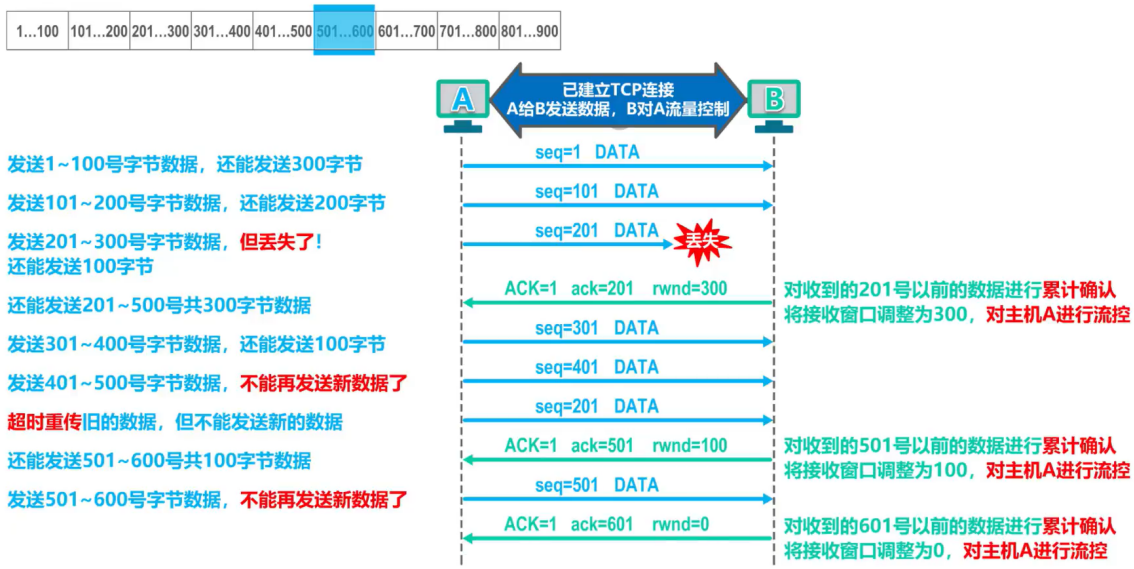
具体流程的视频

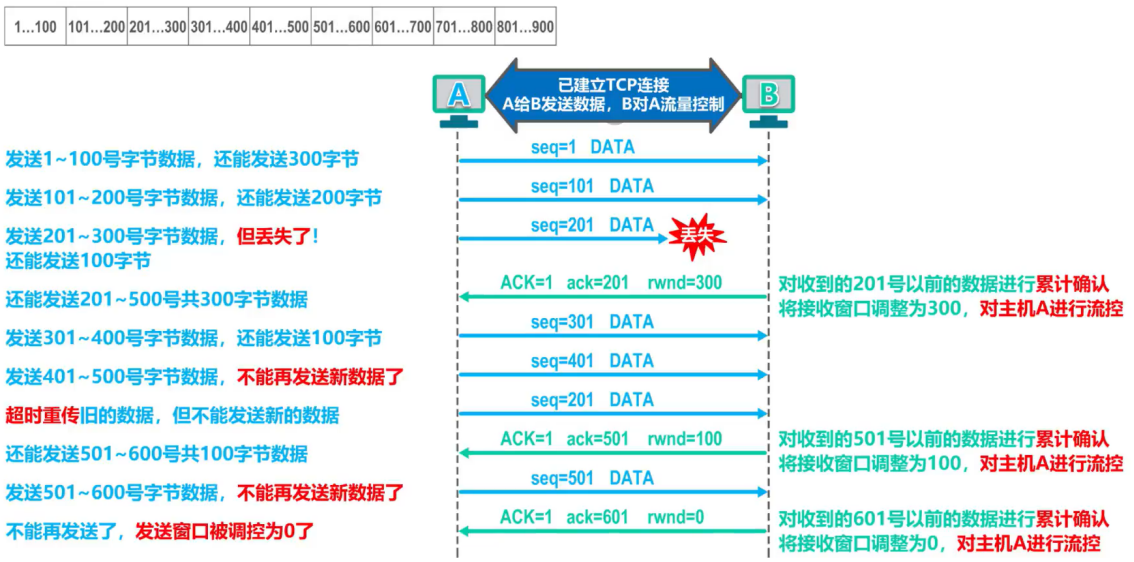


上图主机A现在可将发送缓存中序号1~200的字节数据全部删除，因为已经收到了主机B对它们的累计确认

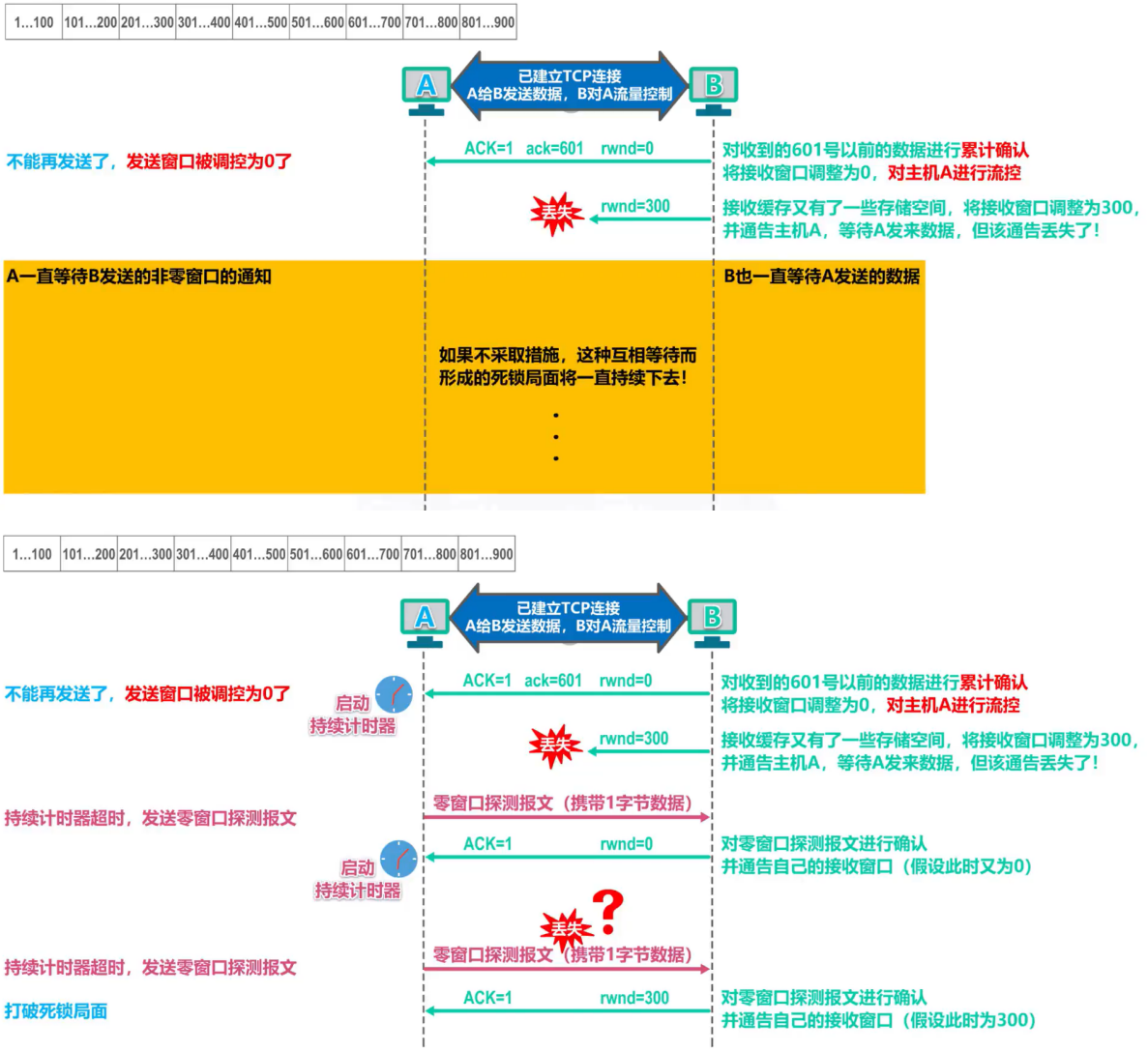


上图主机A现在可将发送缓存中序号201~500的字节数据全部删除，因为已经收到了主机B对它们的累计确认





上图主机A现在可将发送缓存中序号501~600的字节数据全部删除，因为已经收到了主机B对它们的累计确认



> 上图如果零窗口探测报文在发送过程中如果丢失，还是能打破死锁局面
>
> 因为零窗口探测报文段也有重传计时器，重传计时器超时后，零窗口探测报文段会被重传

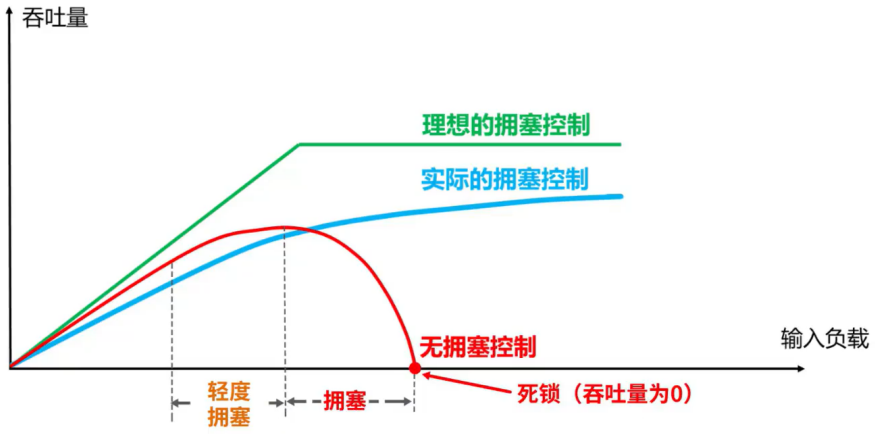
总结

- **一般来说，我们总是希望数据传输得更快一些。**
  - **但如果发送方把数据发送得过快，接收方就可能来不及接收，这就会造成数据的丢失。**
- **所谓流量控制（flow control）就是让发送方的发送速率不要太快，要让接收方来得及接收。**
- **利用滑动窗口机制可以很方便地在TCP连接上实现对发送方的流量控制。**
  - **TCP接收方利用自己的接收窗口的大小来限制发送方发送窗口的大小。**
  - **TCP发送方收到接收方的零窗口通知后，应启动持续计时器。持续计时器超时后，向接收方发送零窗口探测报文。**

5.5、TCP的拥塞控制

概念

- 在某段时间，若**对网络中某一资源的需求超过了该资源所能提供的可用部分，网络性能就要变坏。**这种情况就叫做**拥塞**(congestion)。
  - 在计算机网络中的链路容量（即带宽）、交换结点中的缓存和处理机等，都是网络的资源。
- 若**出现拥塞而不进行控制**，整个网络的**吞吐量将随输入负荷的增大而下降。**



网络拥塞往往是由许多因素引起的。例如：

1. 点缓存的容量太小；
2. 链路的容量不足；
3. 处理机处理的速率太慢；
4. 拥塞本身会进一步加剧拥塞；

拥塞控制的一般原理

- 拥塞控制的前提：网络能够承受现有的网络负荷。
- 实践证明，拥塞控制是很难设计的，因为它是一个**动态问题**。
- 分组的丢失是网络发生拥塞的**征兆**而不是原因。
- 在许多情况下，甚至正是**拥塞控制本身**成为引起网络性能恶化、甚至发生死锁的原因。

开环控制和闭环控制

开环控制

在设计网络时，事先考虑周全，力求工作时不发生拥塞；

思路：力争避免发生拥塞。

闭环控制

基于反馈环路的概念；

根据网络当前的运行状态采取相应控制措施；

思路：在发生拥塞后，采取措施进行控制，消除拥塞。

监测网络的拥塞

主要指标有：

1. 由于缺少缓存空间而被丢弃的分组的百分数；
2. 平均队列长度；
3. 超时重传的分组数；
4. 平均分组时延；
5. 分组时延的标准差，等等。

上述这些指标的上升都标志着拥塞的增长。

拥塞控制的算法

> 真正的发送窗口值 = Min (接收方窗口值，拥塞窗口值)

下图的实例横纵坐标的意思

传输轮次：

- 发送方给接收方发送数据报文段后，接收方给发送方发发回相应的确认报文段
- 一个传输轮次所经历的时间其实就是往返时间，往返时间并非是恒定的数值

- 使用传输轮次是为了强调把拥塞窗口所允许发送的报文段都连续发送出去，并受到了对已发送的最后一个报文段的确认

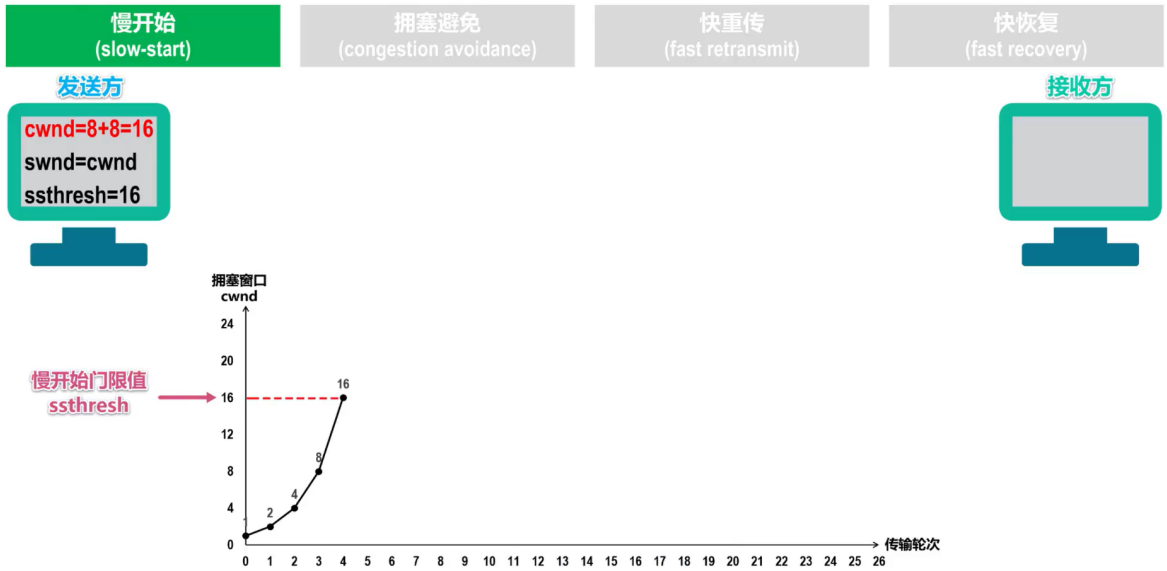
拥塞窗口：

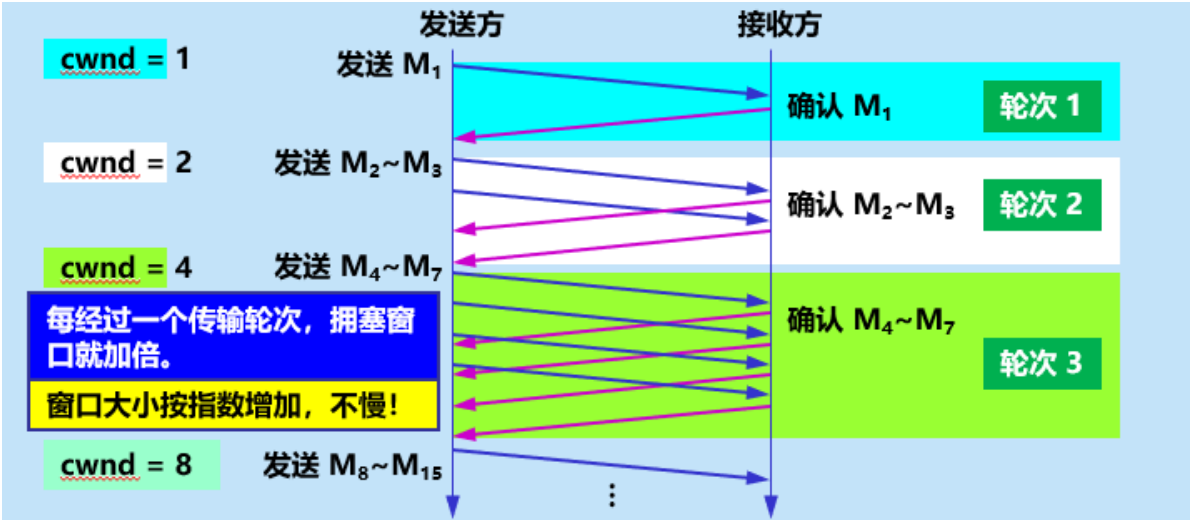
- 它会随网络拥塞程度，以及所使用的拥塞控制算法动态变化

慢开始和拥塞避免

慢开始（slow-start）

- 目的：用来确定网络的负载能力或拥塞程度。
- 算法的思路：由小到大逐渐增大拥塞窗口数值。
- 两个变量：
  - **拥塞窗口（cwnd）**：初始拥塞窗口值：2 种设置方法。窗口值逐渐增大。
    - 1 至 2 个最大报文段 （旧标准）
    - 2 至 4 个最大报文段 （RFC 5681）
  - **慢开始门限（ssthresh）**：防止拥塞窗口增长过大引起网络拥塞。



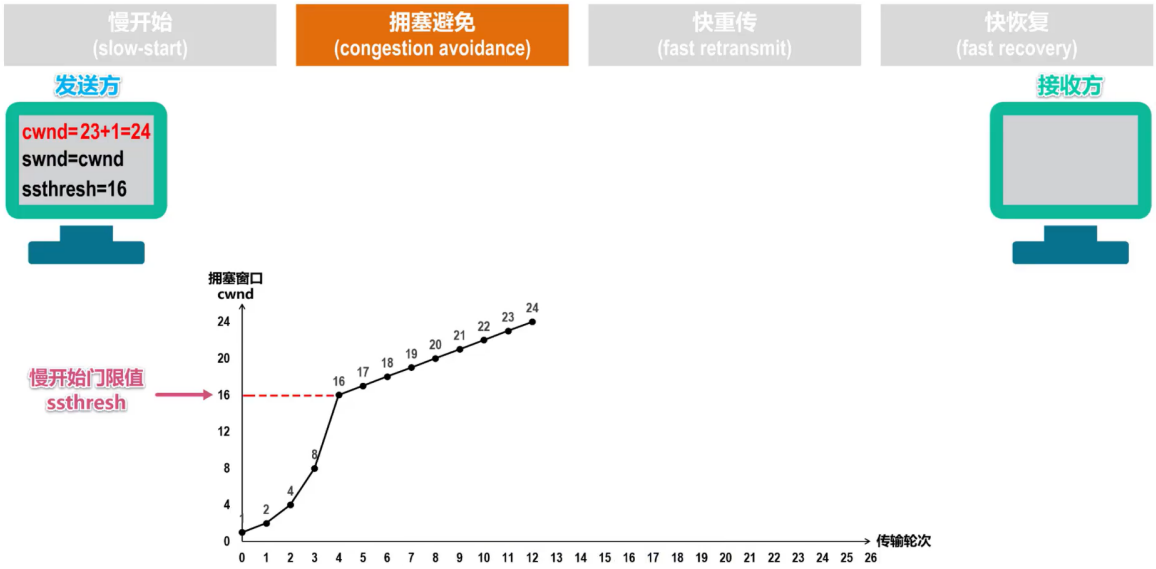


> 图中swnd是发送窗口
>
> 每经过一个传输轮次，拥塞窗口就加倍

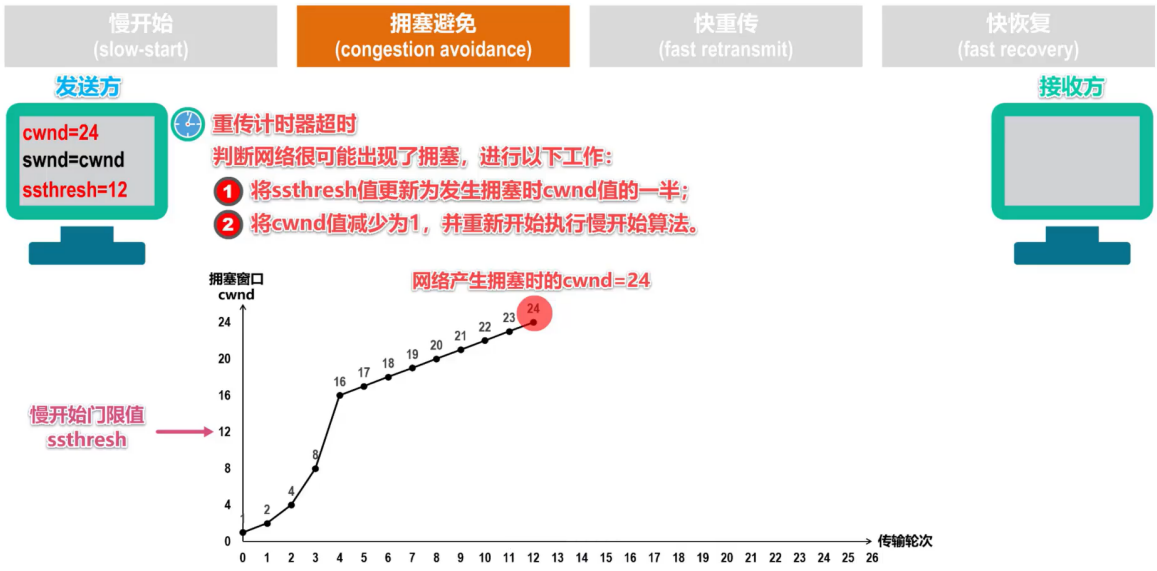
> 窗口大小按**指数增加**，2的n-1次方

拥塞避免（congestion avoidance）

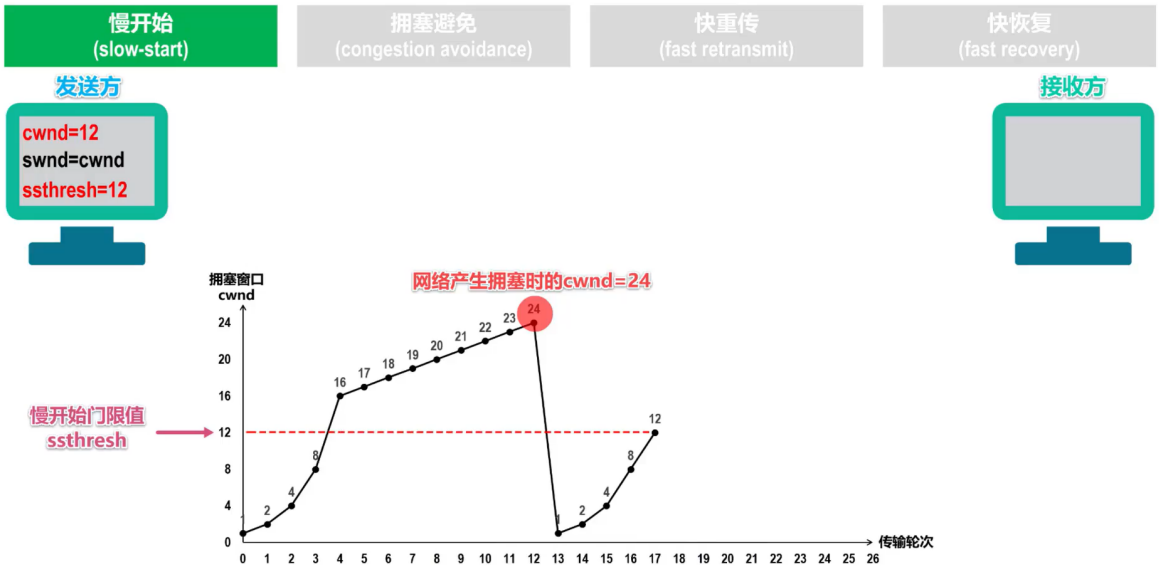
- 思路：让拥塞窗口 cwnd **缓慢地增大**，避免出现拥塞。
- 每经过一个传输轮次，拥塞窗口 **cwnd = cwnd + 1**。
- 使拥塞窗口 cwnd 按线性规律缓慢增长。
- 在拥塞避免阶段，具有“**加法增大**” (Additive Increase) 的特点。



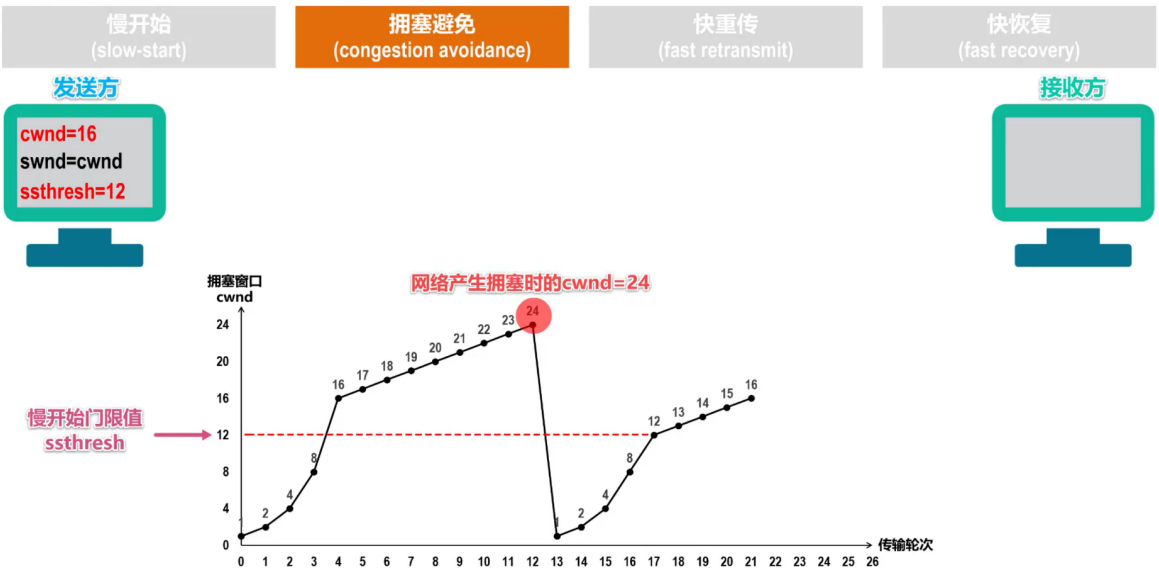
如果在发送过程中出现部分报文段丢失，这必然会造成发送方对这些丢失报文段的超时重传



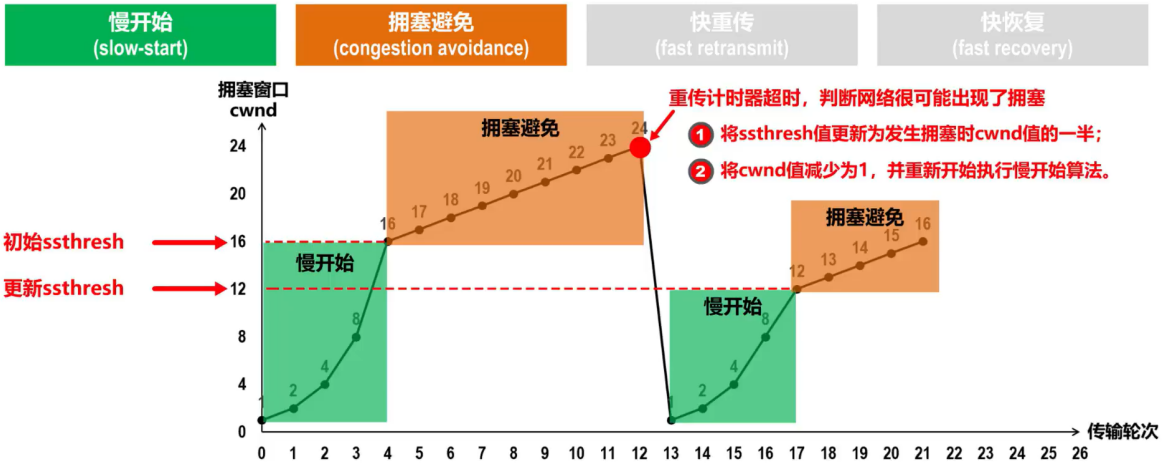
这个时候又回到了慢开始



5.5 TCP的拥塞控制



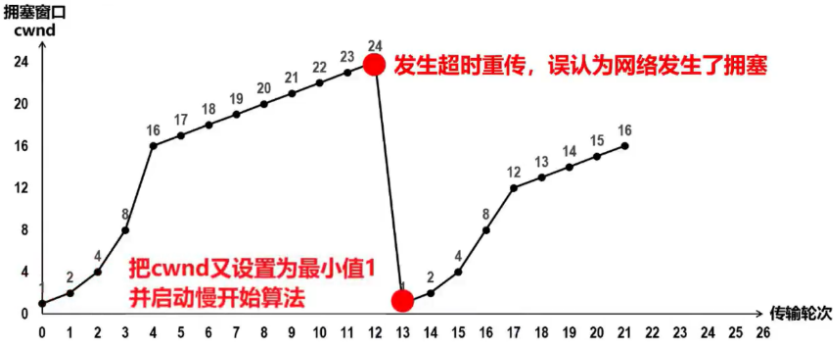
两个算法完整示意图



- “慢开始”是指一开始向网络注入的报文段少，并不是指拥塞窗口cwnd增长速度慢；
- “拥塞避免”并非指完全能够避免拥塞，而是指在拥塞避免阶段将拥塞窗口控制为按线性规律增长，使网络比较不容易出现拥塞；

快重传和快恢复

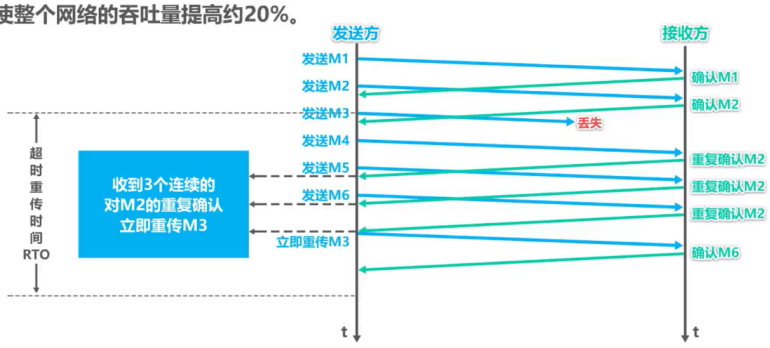
- 慢开始和拥塞避免算法是1988年提出的TCP拥塞控制算法（TCP Tahoe版本）。
- 1990年又增加了两个新的拥塞控制算法(**改进TCP的性能**)，这就是快重传和快恢复（TCP Reno版本）。
  - 有时，**个别报文段**会在网络中**丢失**，但实际上网络**并未发生拥塞**。
    - 这将导致**发送方**超时重传，并**误认为**网络发生了**拥塞**；
    - **发送方把拥塞窗口cwnd又设置为最小值1**，并错误地启动慢开始算法，因而**降低了传输效率**。



- 采用快重传算法可以**让发送方尽早知道发生了个别报文段的丢失**。
- 所谓快重传，就是使发送方**尽快进行重传**，而**不是等超时重传计时器超时**再重传。
  - 要求接收方不要等待自己发送数据时才进行捎带确认，而是要**立即发送确认**；
  - 即使收到了失序的报文段也要立即发出对已收到的报文段的**重复确认**。
  - 发送方一旦**收到3个连续的重复确认**，就将相应的报文段**立即重传**，而不是等该报文段的超时重传计时器超时再重传。

快重传（fast retrasmit）

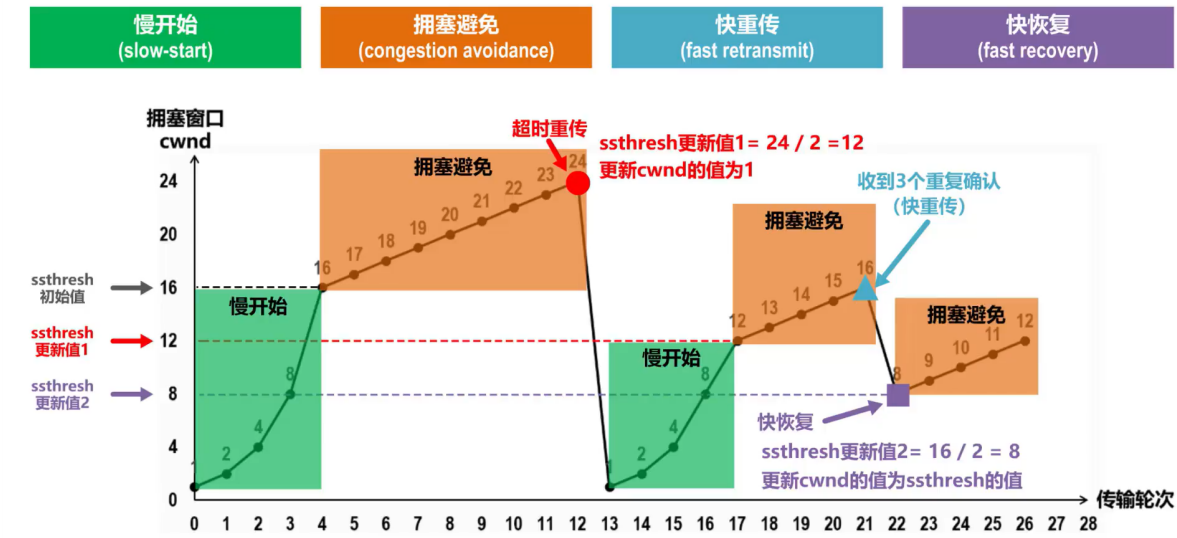
- 所谓快重传，就是使发送方**尽快进行重传**，而**不是等超时重传计时器超时**再重传。
  - 要求接收方不要等待自己发送数据时才进行捎带确认，而是要**立即发送确认**；
  - 即使收到了失序的报文段也要立即发出对已收到的报文段的**重复确认**。
  - 发送方一旦**收到3个连续的重复确认**，就将相应的报文段**立即重传**，而不是等该报文段的超时重传计时器超时再重传。
  - 对于个别丢失的报文段，发送方不会出现超时重传，也就不会误认为出现了拥塞（进而降低拥塞窗口cwnd为1）。使用快重传可以使整个网络的吞吐量提高约20%。



快恢复（fast recovery）

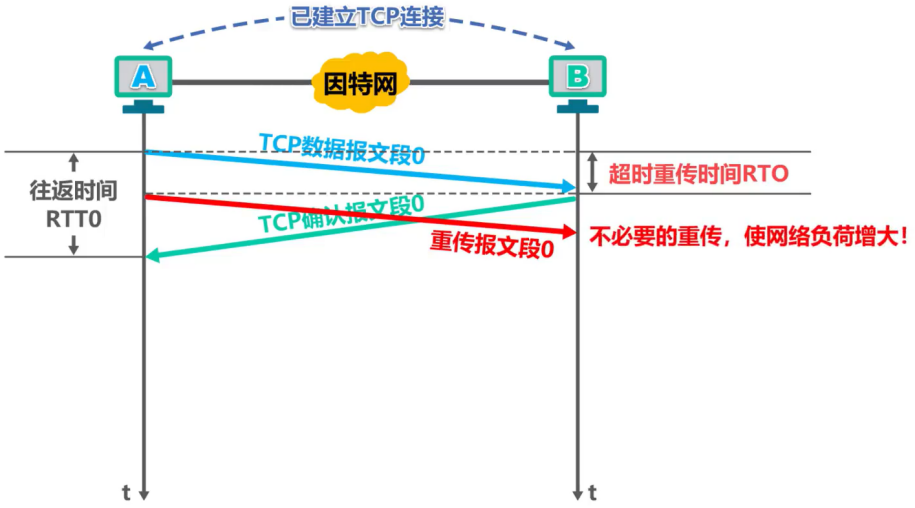
- 发送方一旦**收到3个重复确认**，就知道现在只是丢失了个别的报文段。于是不启动慢开始算法，而**执行快恢复算法**；
  - **发送方将慢开始门限ssthresh值和拥塞窗口cwnd值调整为当前窗口的一半；开始执行拥塞避免算法。**
  - 也有的快恢复实现是把快恢复开始时的拥塞窗口cwnd值再增大一些，即等于新的ssthresh + 3。
    - 既然发送方收到3个重复的确认，就表明有3个数据报文段已经离开了网络；
    - 这3个报文段不再消耗网络资源而是停留在接收方的接收缓存中；
    - 可见现在网络中不是堆积了报文段而是减少了3个报文段。因此可以适当把拥塞窗口扩大些。

改进后的整体算法的示意图



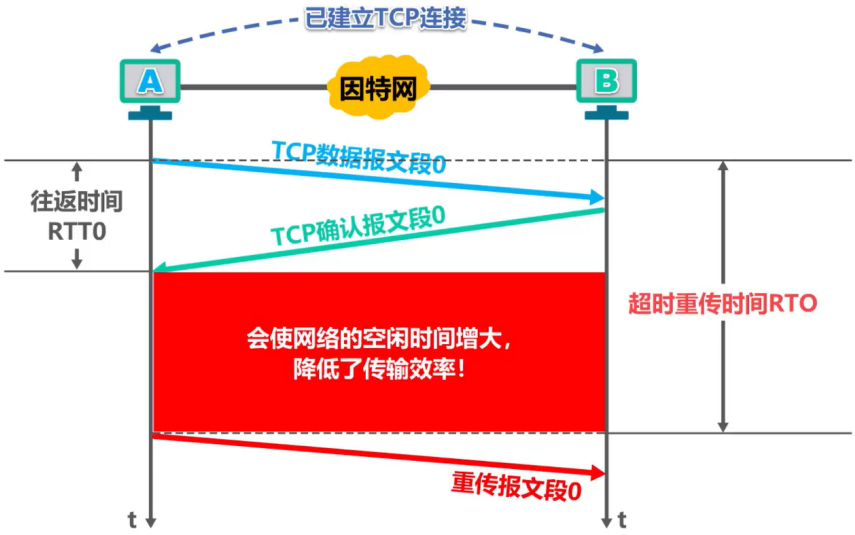
5.6、 TCP超时重传时间的选择

■ 超时重传时间的选择是TCP最复杂的问题之一

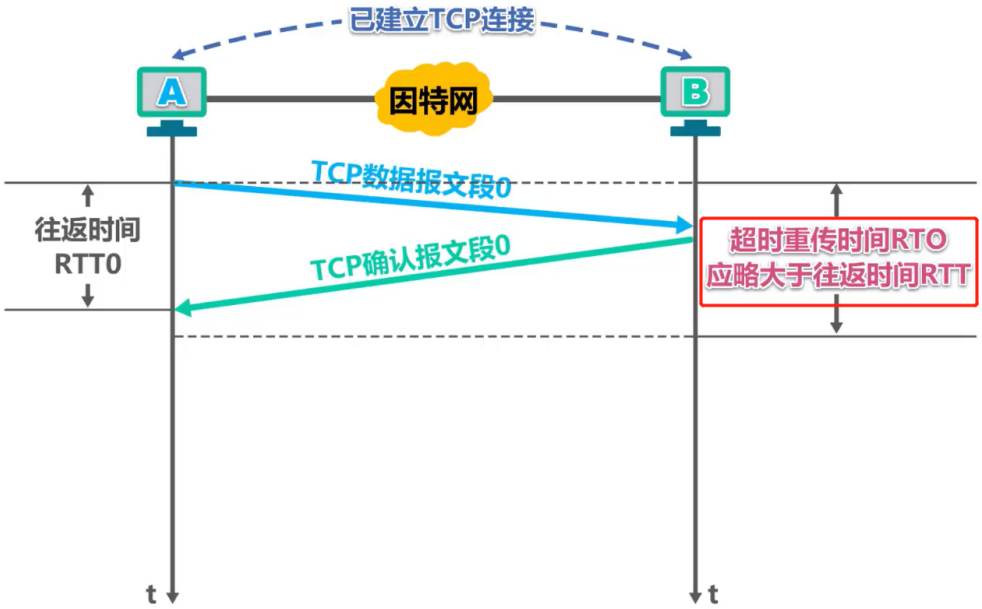


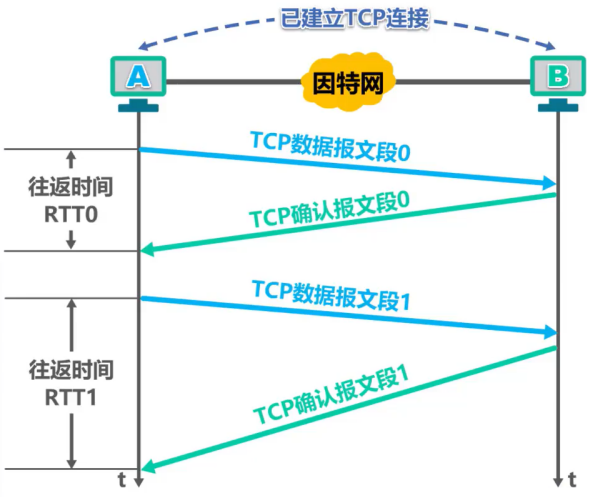
> 如果超时重传时间RTO的值设置得比RTT0的值小很多，这会引起报文段不必要的重传，使网络负荷增大

■ 超时重传时间的选择是TCP最复杂的问题之一



> 如果超时重传时间RTO的值设置得远大于RTT0的值，这会使重传时间推迟的太长，使网络的空闲时间增大，降低传输效率





■ **不能直接使用某次测量得到的RTT样本来计算超时重传时间RTO。**

■ 利用每次测量得到的RTT样本，计算加权平均**往返时间RTTs**（又称为平滑的往返时间）。

$RTT_{S1} = RTT_1$

新的 $RTT_S = (1 - \alpha) \times$ 旧的 $RTT_S + \alpha \times$ 新的 RTT 样本

在上式中，$0 \leq \alpha < 1$ ：

若 α 很接近于0，则新RTT样本对RTTs的影响不大；

若 α 很接近于1，则新RTT样本对RTTs的影响较大；

已成为建议标准的RFC6298推荐的 α 值为1/8，即0.125。

■ 用这种方法得出的加权平均往返时间RTTs就比测量出的RTT值更加平滑。

■ 显然，**超时重传时间RTO应略大于加权平均往返时间RTTs。**

RFC6298建议使用下式计算超时重传时间RTO

$$RTO = RTT_S + 4 \times RTT_D$$

加权平均往返时间RTTs

$RTT_{S1} = RTT_1$

新的 $RTT_S = (1 - \alpha) \times$ 旧的RTT_S

$+ \alpha \times$ 新的RTT样本

$0 \le \alpha < 1$

已成为建议标准的RFC6298推荐的α值为1/8，即0.125。

RTT偏差的加权平均RTTD

$RTT_{D1} = RTT_1 \div 2$

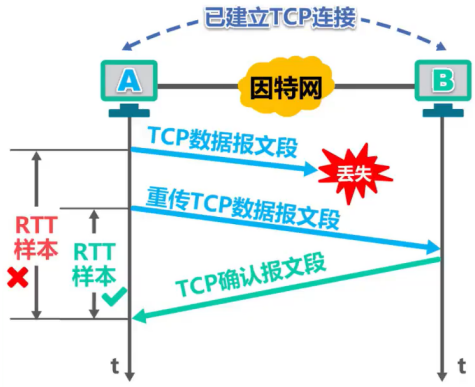
新的 $RTT_D = (1 - \beta) \times$ 旧的RTT_D

$+ \beta \times |RTT_S -$ 新的RTT样本$|$

$0 \le \beta < 1$

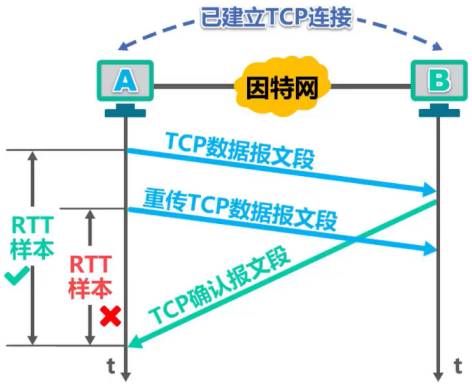
已成为建议标准的RFC6298推荐的β值为1/4，即0.25。

往返时间RTT的测量比较复杂



源主机若误将确认当作是对原报文段的确认：

所计算出的RTTs和RTO就会偏大，降低了传输效率；



源主机若误将确认当作是对重传报文段的确认：

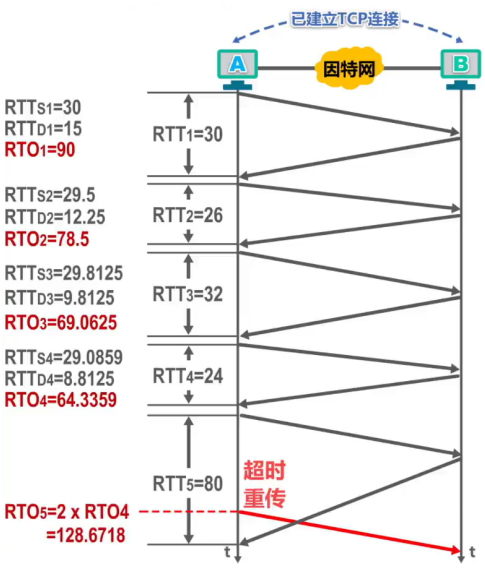
所计算出的RTTs和RTO就会偏小，导致报文段没必要的重传，增大网络负荷；

- 针对出现超时重传时无法测准往返时间RTT的问题，Karn提出了一个算法：**在计算加权平均往返时间RTTs时，只要报文段重传了，就不采用其往返时间RTT样本**。也就是出现重传时，不重新计算RTTs，进而超时重传时间RTO也不会重新计算。
  - 这又引起了新的问题。设想出现这样的情况：报文段的时延突然增大了很多，并且之后很长一段时间都会保持这种时延。因此在原来得出的重传时间内，不会收到确认报文段。于是就重传报文段。但根据Karn算法，不考虑重传的报文段的往返时间样本。这样，超时重传时间就无法更新。这会导致报文段反复被重传。

- 因此，要对Karn算法进行修正。方法是：**报文段每重传一次，就把超时重传时间RTO增大一些**。典型的做法是将新RTO的值取为旧RTO值的2倍。

TCP超时重传的计算

举例



$$RTO = RTT_S + 4 \times RTT_D$$

$RTT_{S1} = RTT_1$

新的 $RTT_S = (1 - \alpha) \times$ 旧的 $RTT_S + \alpha \times$ 新的 RTT 样本

$\alpha = 0.125$

$RTT_{D1} = RTT_1 \div 2$

新的 $RTT_D = (1 - \beta) \times$ 旧的 $RTT_D + \beta \times |RTT_S -$ 新的 RTT 样本 $|$

$\beta = 0.25$

出现超时重传时，新RTO=2倍的旧RTO

总结

$$RTO = RTT_S + 4 \times RTT_D$$

加权平均往返时间RTTs

$RTT_{S1} = RTT_1$

新的 $RTT_S = (1 - \alpha) \times$ 旧的 $RTT_S + \alpha \times$ 新的 RTT 样本

$0 \le \alpha < 1$

已成为建议标准的RFC6298推荐的 α 值为1/8，即0.125。

RTT偏差的加权平均RTTD

$RTT_{D1} = RTT_1 \div 2$

新的 $RTT_D = (1 - \beta) \times$ 旧的 $RTT_D + \beta \times |RTT_S -$ 新的 RTT 样本 $|$

$0 \le \beta < 1$

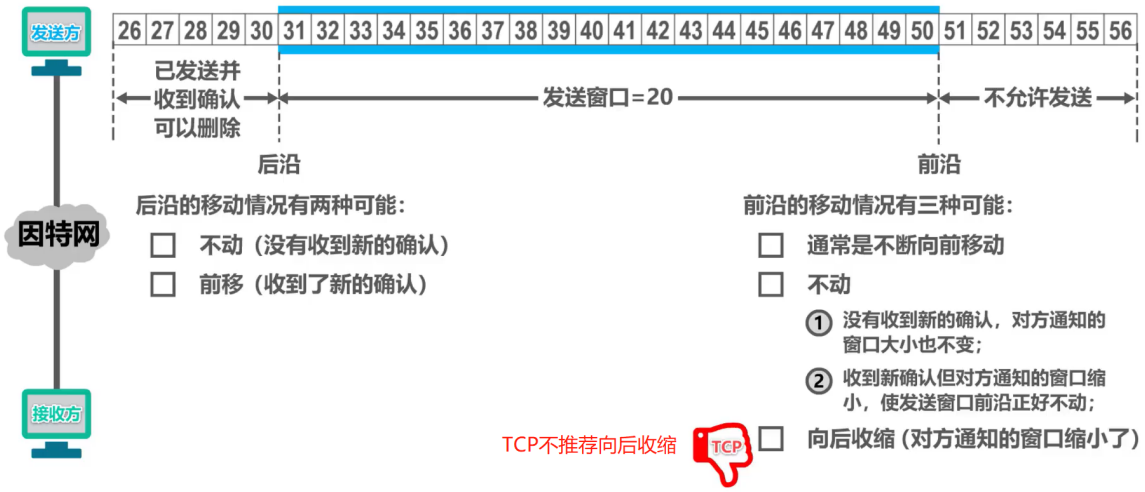
已成为建议标准的RFC6298推荐的 β 值为1/4，即0.25。

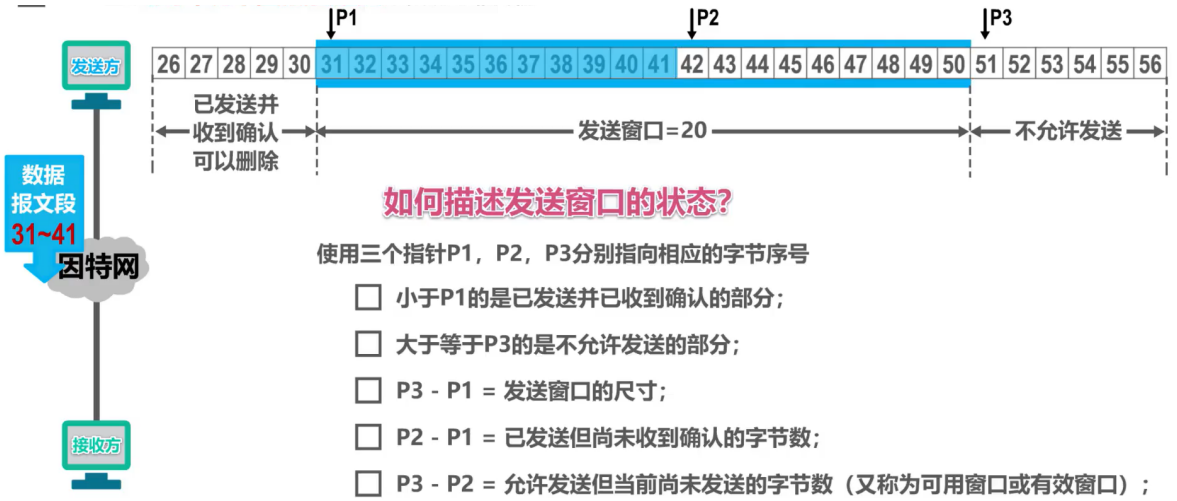
出现超时重传时，新RTO=2倍的旧RTO

5.7、TCP可靠传输的实现

本集具体讲解

■ TCP基于**以字节为单位的滑动窗口**来实现可靠传输





■ TCP基于**以字节为单位的滑动窗口**来实现可靠传输。
- □ 发送方在未收到接收方的确认时，可将发送窗口内还未发送的数据全部发送出去；
- □ 接收方只接收序号落入发送窗口内的数据；

■ 虽然发送方的发送窗口是根据接收方的接收窗口设置的，但在同一时刻，**发送方的发送窗口并不总是和接收方的接收窗口一样大。**
- □ 网络传送窗口值需要经历一定的时间滞后，并且这个时间还是不确定的。
- □ 发送方还可能根据网络当时的拥塞情况适当减小自己的发送窗口尺寸。

■ 对于**不按序到达的数据应如何处理**，TCP并无明确规定。
- □ 如果接收方把不按序到达的数据一律丢弃，那么接收窗口的管理将会比较简单，但这样做对网络资源的利用不利，因为发送方会重复传送较多的数据。
- □ TCP通常对不按序到达的数据是先临时存放在接收窗口中，等到字节流中所缺少的字节收到后，再**按序交付上层的应用进程**。

■ TCP要求接收方必须有**累积确认和捎带确认机制**，这样可以减小传输开销。接收方可以在合适的时候发送确认，也可以在自己有数据要发送时把确认信息顺便捎带上。
- □ **接收方不应过分推迟发送确认**，否则会导致发送方不必要的超时重传，这反而浪费了网络的资源。

  TCP标准规定，确认推迟的时间不应超过0.5秒。若收到一连串具有最大长度的报文段，则必须每隔一个报文段就发送一个确认[RFC 1122]。
- □ 捎带确认实际上并不经常发生，因为大多数应用程序很少同时在两个方向上发送数据。

■ **TCP的通信是全双工通信。**通信中的每一方都在发送和接收报文段。因此，每一方都有自己的发送窗口和接收窗口。在谈到这些窗口时，一定要弄清楚是哪一方的窗口。

5.8、 TCP的运输连接管理

概念

- TCP是面向连接的协议，它基于运输连接来传送TCP报文段。
- TCP运输连接的建立和释放是每一次面向连接的通信中必不可少的过程。
- TCP运输连接有以下三个阶段：

① 建立TCP连接

② 数据传送

③ 释放TCP连接

- TCP的运输连接管理就是使运输连接的建立和释放都能正常地进行。

TCP的连接建立

- TCP 建立连接的过程叫做**握手**。
- 握手需要在客户和服务器之间交换三个 TCP 报文段。称之为**三报文握手**。
- 采用**三报文握手**主要是为了防止已失效的连接请求报文段突然又传送到了，因而产生错误。

TCP的连接建立要解决以下三个问题

- TCP的连接建立要解决以下三个问题：

① 使TCP双方能够确知对方的存在；

② 使TCP双方能够协商一些参数（如最大窗口值、是否使用窗口扩大选项和时间戳选项以及服务质量等）；

③ 使TCP双方能够对运输实体资源（如缓存大小、连接表中的项目等）进行分配。

TCP使用“三报文握手”建立连接

- TCP 连接的建立**采用客户服务器方式**。
- 主动发起连接建立的应用进程叫做**TCP客户** (client)。
- 被动等待连接建立的应用进程叫做**TCP服务器** (server)。

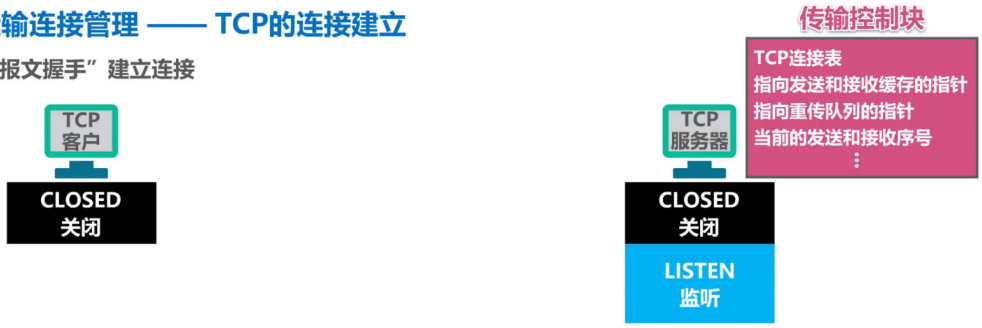
“握手”需要在TCP客户端和服务器之间交换三个TCP报文段

过程

> 最初两端的TCP进程都处于关闭状态

5.8.1 TCP的运输连接管理 —— TCP的连接建立

■ TCP使用“三报文握手”建立连接

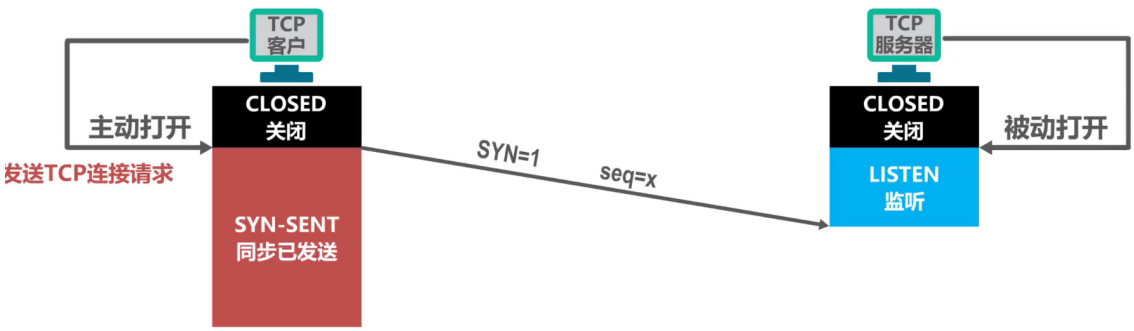


> 一开始，TCP服务器进程首先创建传输控制块，用来存储TCP连接中的一些重要信息。例如TCP连接表、指向发送和接收缓存的指针、指向重传队列的指针，当前的发送和接收序号等
>
> 之后，就准备接受TCP客户端进程的连接请求
>
> 此时，TCP服务器进程就进入监听状态，等待TCP客户端进程的连接请求

TCP服务器进程是被动等待来自TCP客户端进程的连接请求，因此成为**被动打开**连接

> TCP客户进程也是首先创建传输控制块

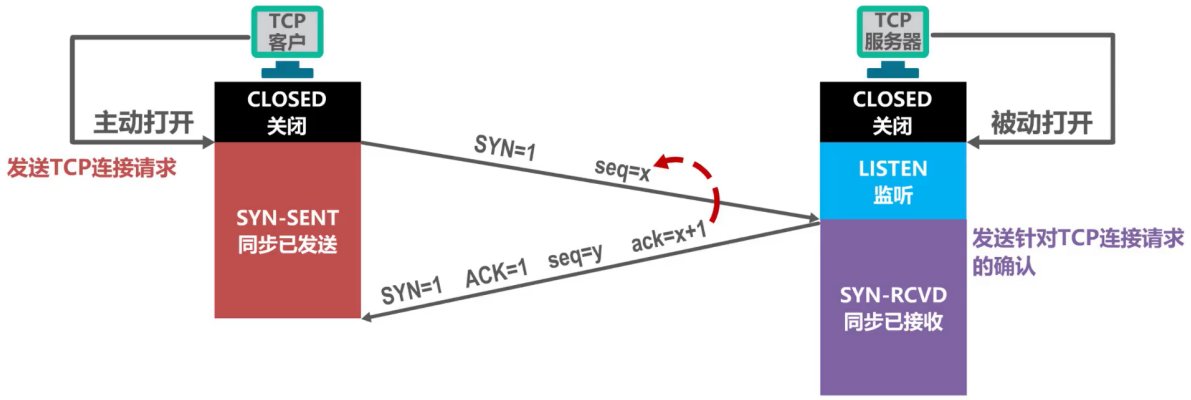
由于TCP连接建立是由TCP客户端主动发起的，因此称为**主动打开**连接



> 然后，在打算建立TCP连接时，向TCP服务器进程发送TCP连接请求报文段，并进入同步已发送状态
>
> TCP连接请求报文段首部中
>
> - 同步位SYN被设置为1，表明这是一个TCP连接请求报文段

- 序号字段seq被设置了一个初始值x，作为TCP客户端进程所选择的初始序号

请注意：TCP规定SYN被设置为1的报文段不能携带数据，但要消耗掉一个序号

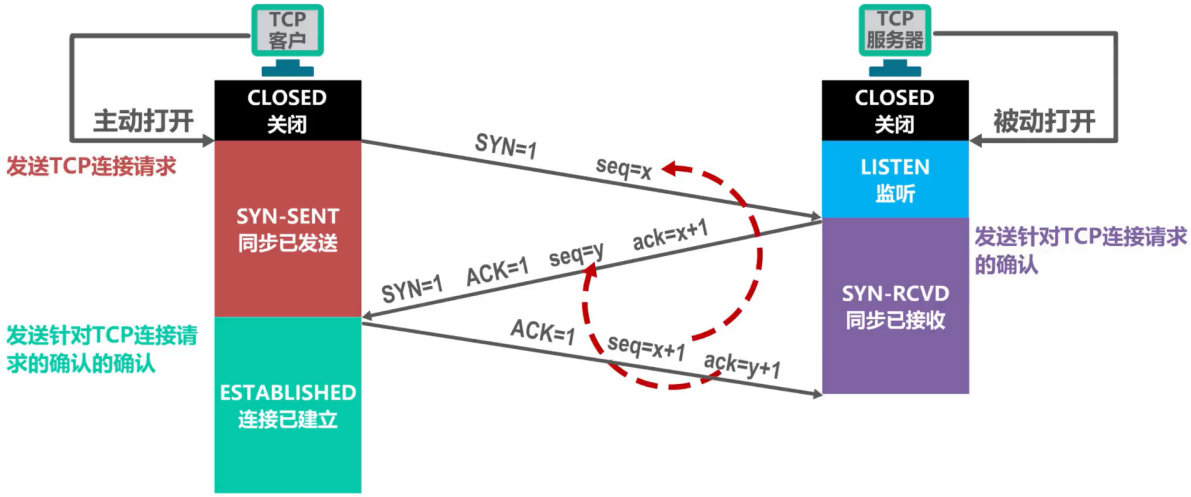


TCP服务器进程收到TCP连接请求报文段后，如果同意建立连接，则向TCP客户进程发送TCP连接请求确认报文段，并进入同步已接收状态

TCP连接请求确认报文段首部中

- 同步位SYN和确认位ACK都设置为1，表明这是一个TCP连接请求确认报文段
- 序号字段seq被设置了一个初始值y，作为TCP服务器进程所选择的初始序号，
- 确认号字段ack的值被设置成了x+1，这是对TCP客户进程所选择的初始序号（seq）的确认

请注意：这个报文段也不能携带数据，因为它是SYN被设置为1的报文段，但同样要消耗掉一个序号

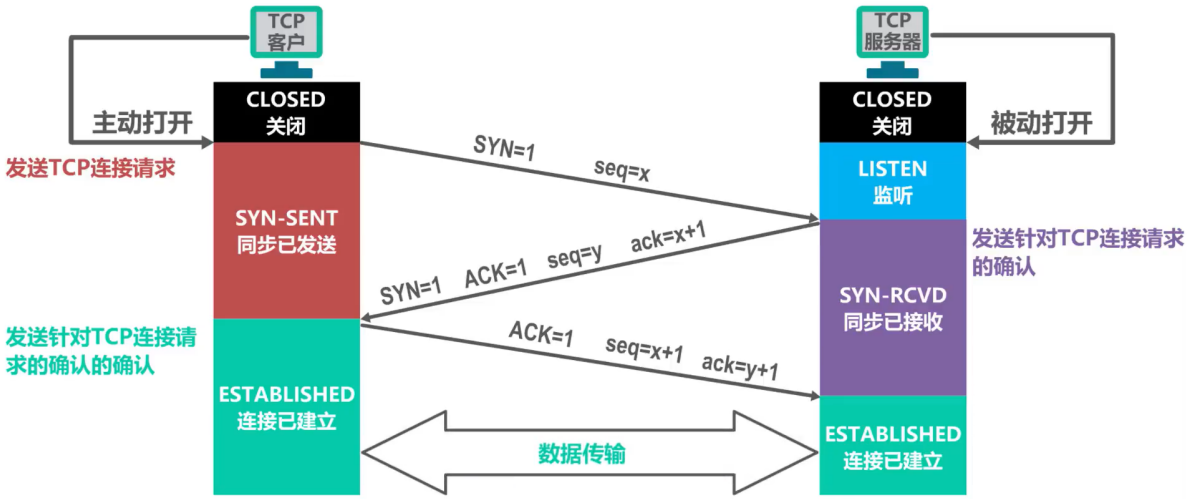


TCP客户进程收到TCP连接请求确认报文段后，还要向TCP服务器进程发送一个普通的TCP确认报文段，并进入连接已连接状态

普通的TCP确认报文段首部中

- 确认位ACK被设置为1，表明这是一个普通的TCP确认报文段
- 序号字段seq被设置为x+1，这是因为TCP客户进程发送的第一个TCP报文段的序号为x，所以TCP客户进程发送的第二个报文段的序号为x+1
- 确认号字段ack被设置为y+1，这是对TCP服务器进程所选择的初始序号的确认

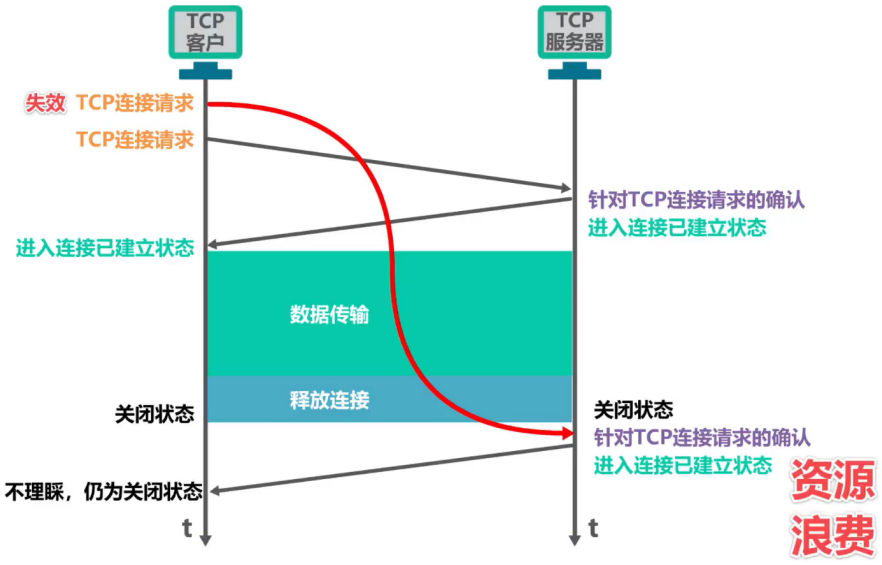
请注意：TCP规定普通的TCP确认报文段可以携带数据，但如果不携带数据，则不消耗序号



> TCP服务器进程收到该确认报文段后也进入连接已建立状态
>
> 现在，TCP双方都进入了连接已建立状态，它们可以基于已建立好的TCP连接，进行可靠的数据传输

为什么TCP客户进程最后还要发送一个普通的TCP确认报文段？能否使用“两报文握手”建立连接？

下图实例是“两报文握手”



> 为了防止已经失效的连接请求报文段突然又传到服务端，因而产生错误”，这种情况是：一端(client)A发出去的第一个连接请求报文并没有> 丢失，而是因为某些未知的原因在某个网络节点上发生滞留，导致延迟到连接释放以后的某个时间才到达另一端(server)B。本来这是一个> 早已失效的报文段，但是B收到此失效的报文之后，会误认为是A再次发出的一个新的连接请求，于是B端就向A又发出确认报文，表示同> 意建立连接。如果不采用“三次握手”，那么只要B端发出确认报文就会认为新的连接已经建立了，但是A端并没有发出建立连接的请求，因> 此不会去向B端发送数据，B端没有收到数据就会一直等待，这样B端就会白白浪费掉很多资源。
>
> 所以并不**多余**，这是为了防止已失效的连接请求报文段突然又传送到了TCP服务器，因而导致错误

总结

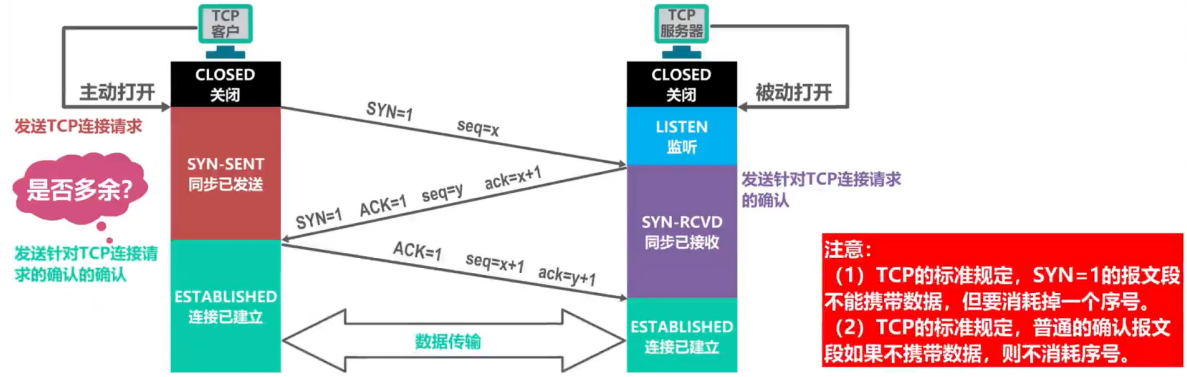
■ TCP的连接建立要解决以下三个问题：

① 使TCP双方能够确知对方的存在；

② 使TCP双方能够协商一些参数（如最大窗口值、是否使用窗口扩大选项和时间戳选项以及服务质量等）；

③ 使TCP双方能够对运输实体资源（如缓存大小、连接表中的项目等）进行分配。

■ TCP使用“三报文握手”建立连接



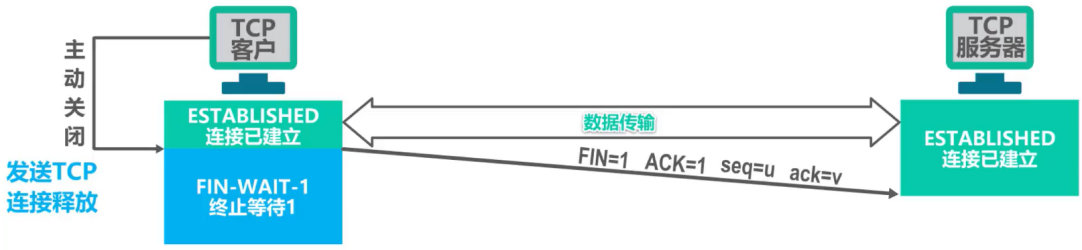
TCP的连接释放

- TCP 连接释放过程比较复杂。
- 数据传输结束后，通信的双方都可释放连接。
- TCP 连接释放过程是**四报文握手**。

TCP通过“四报文挥手”来释放连接

- TCP 连接的建立**采用客户服务器方式**。
- 主动发起连接建立的应用进程叫做**TCP客户** (client)。
- 被动等待连接建立的应用进程叫做**TCP服务器** (server)。
- **任何一方都可以在数据传送结束后发出连接释放的通知**

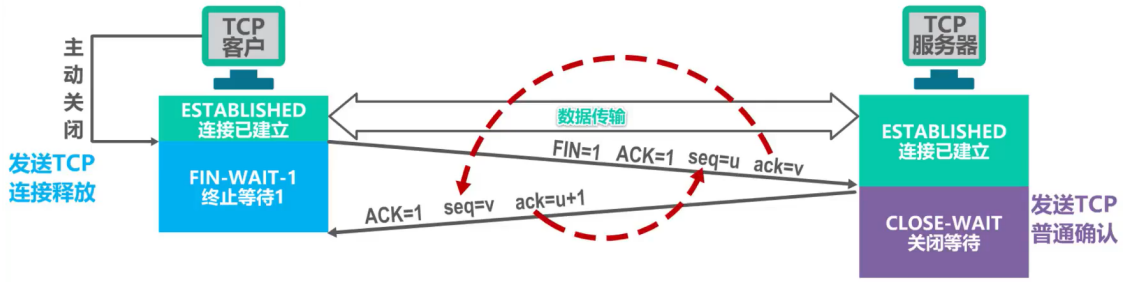
过程



> 现在TCP客户进程和TCP服务器进程都处于连接已建立状态
>
> TCP客户进程的应用进程通知其主动关闭TCP连接
>
> TCP客户进程会发送TCP连接释放报文段，并进入终止等待1状态
>
> TCP连接释放报文段首部中

- 终止位FIN和确认为ACK的值都被设置为1，表明这是一个TCP连接释放报文段，同时也对之前收到的报文段进行确认
- 序号seq字段的值设置为u，它等于TCP客户进程之前已传送过的数据的最后一个字节的序号加1
- 确认号ack字段的值设置为v，它等于TCP客户进程之前已收到的、数据的最后一个字节的序号加1

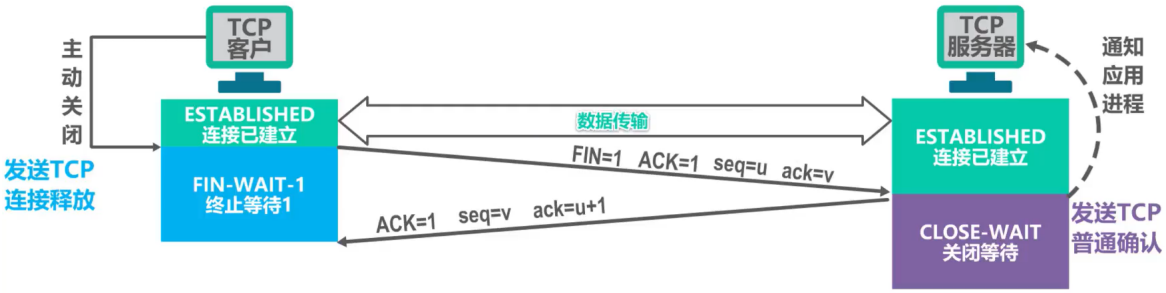
请注意：TCP规定终止位FIN等于1的报文段即使不携带数据，也要消耗掉一个序号



TCP服务器进程收到TCP连接释放报文段后，会发送一个普通的TCP确认报文段并进入关闭等待状态

普通的TCP确认报文段首部中

- 确认位ACK的值被设置为1，表明这是一个普通的TCP确认报文段
- 序号seq字段的值设置为v，它等于TCP服务器进程之前已传送过的数据的最后一个字节的序号加1，这也与之前收到的TCP连接释放报文段中的确认号匹配
- 确认号ack字段的值设置为u+1，这是对TCP连接释放报文段的确认

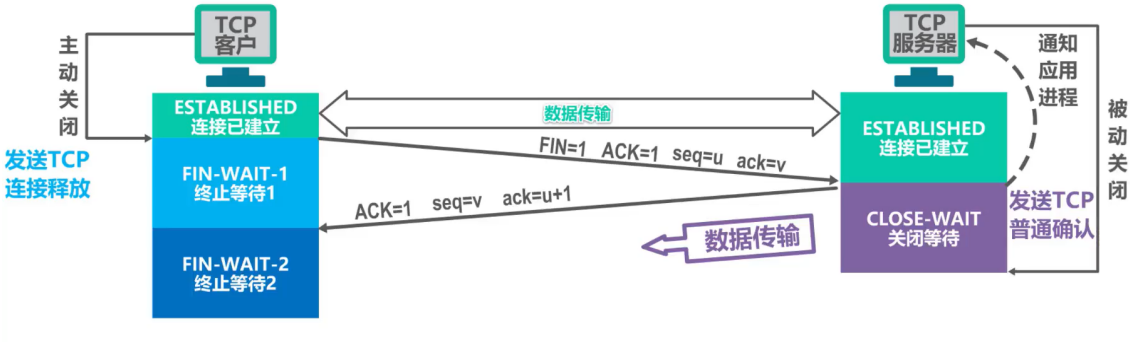


TCP服务器进程应该通知高层应用进程，TCP客户进程要断开与自己的TCP连接

此时，从TCP客户进程到TCP服务器进程这个方向的连接就释放了

这时的TCP连接属于半关闭状态，也就是TCP客户进程已经没有数据要发送了

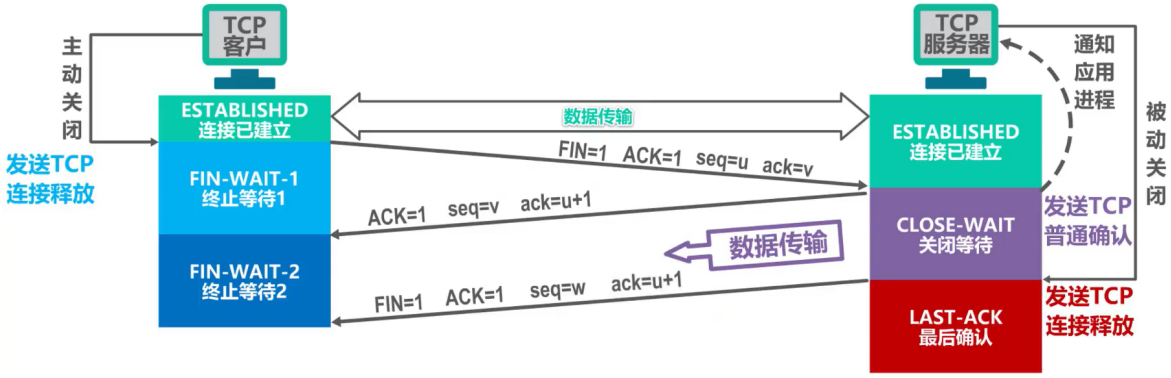
但如果TCP服务器进程还有数据要发送，TCP客户进程仍要接收，也就是说从TCP服务器进程到TCP客户进程这个方向的连接并未关闭



TCP客户进程收到TCP确认报文段后就进入终止等待2状态，等待TCP服务器进程发出的TCP连接释放报文段

若使用TCP服务器进程的应用进程已经没有数据要发送了，应用进程就通知其TCP服务器进程释放连接

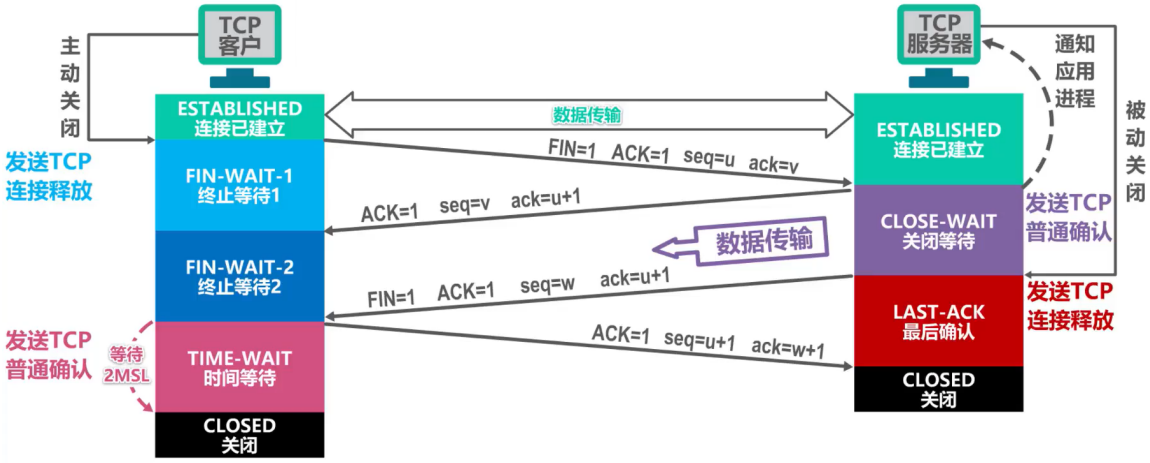
由于TCP连接释放是由TCP客户进程主动发起的，因此TCP服务器进程对TCP连接的释放称为被动关闭连接



TCP服务器进程发送TCP连接释放报文段并进入最后确认状态

该报文段首部中

- 终止位FIN和确认位ACK的值都被设置为1，表明这是一个TCP连接释放报文段，同时也对之前收到的报文段进行确认
- 序号seq字段的值为w，这是因为在半关闭状态下，TCP服务器进程可能又发送
- 确认号ack字段的值为u+1，这是对之前收到的TCP连接释放报文段的重复确认



MSL(Maximum Segment Lifetime)意思是最长报文段寿命，RFC793建议为2分钟。

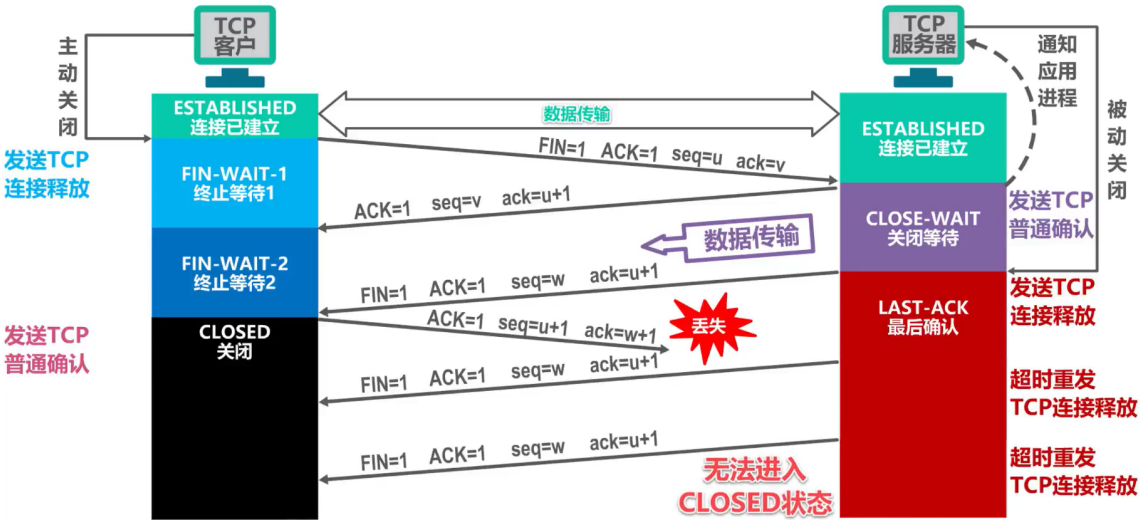
TCP客户进程收到TCP连接释放报文段后，必须针对该报文段发送普通的TCP确认报文段，之后进入时间等待状态

该报文段首部中

- 确认为ACK的值被设置为1，表明这是一个普通的TCP确认报文段
- 序号seq字段的值设置为u+1，这是因为TCP客户进程之前发送的TCP连接释放报文段虽然不携带数据，但要消耗掉一个序号
- 确认号ack字段的值设置为w+1，这是对所收到的TCP连接释放报文段的确认

TCP服务器进程收到该报文段后就进入关闭状态，而TCP客户进程还要进过2MSL后才能进入关闭状态

TCP客户进程在发送完最后一个确认报文后，为什么不直接进入关闭状态？而是要进入时间等待状态？



> 因为时间等待状态以及处于该状态2MSL时长，可以确保TCP服务器进程可以收到最后一个TCP确认报文段而进入关闭状态
>
> 另外，TCP客户进程在发送完最后一个TCP确认报文段后，在经过2MSL时长，就可以使本次连接持续时间内所产生的所有报文段都从网络中消失，这样就可以使下一个新的TCP连接中，不会出现旧连接中的报文段

TCP保活计时器的作用

TCP双方已经建立了连接，后来，TCP客户进程所在的主机突然出现了故障

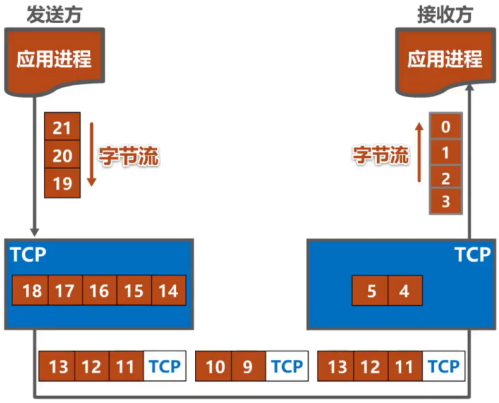
TCP服务器进程以后就不能再收到TCP客户进程发来的数据

因此，应当有措施使TCP服务器进程不要再白白等待下去

- TCP服务器进程每收到一次TCP客户进程的数据，就重新设置并启动**保活计时器**（2小时定时）。
- 若保活计时器定时周期内未收到TCP客户进程发来的数据，则**当保活计时器到时后，TCP服务器进程就向TCP客户进程发送一个探测报文段**，以后则每隔75秒钟发送一次。若一连发送10个探测报文段后仍无TCP客户进程的响应，TCP服务器进程就认为TCP客户进程所在主机出了故障，接着就关闭这个连接。

5.9、 TCP报文段的首部格式

- 为了实现可靠传输，TCP采用了**面向字节流**的方式。
- 但TCP在发送数据时，是从发送缓存取出一部分或全部字节并给其添加一个首部使之成为**TCP报文段**后进行发送。
  - 一个TCP报文段由**首部**和**数据载荷**两部分构成；
  - TCP的全部功能都体现在它首部中各字段的作用。

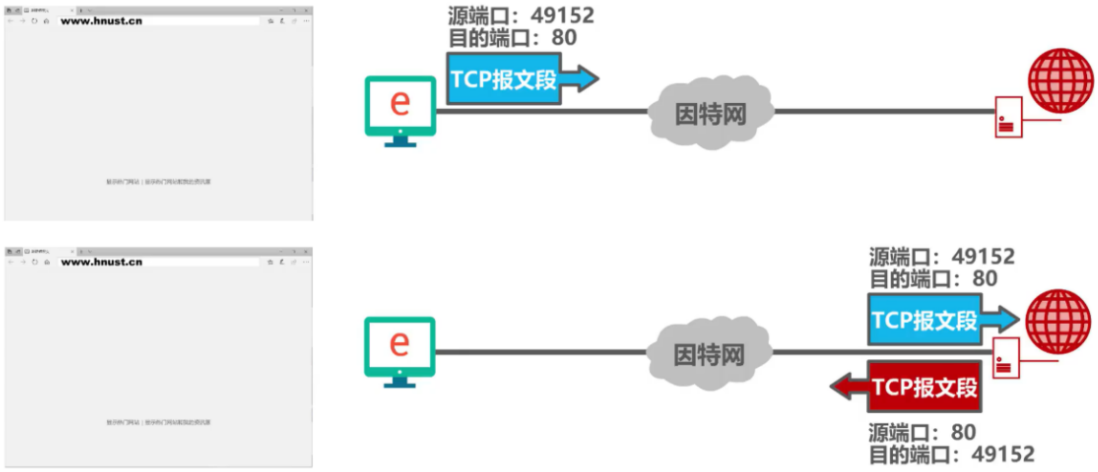


各字段的作用

源端口和目的端口

源端口：占16比特，写入源端口号，用来**标识发送该TCP报文段的应用进程。**

目的端口：占16比特，写入目的端口号，用来**标识接收该TCP报文段的应用进程。**



序号、确认号和确认标志位

序号：占32比特，取值范围[0, $2^{32}-1$]，序号增加到最后一个后，下一个序号就又回到0。

指出本TCP报文段数据载荷的第一个字节的序号。

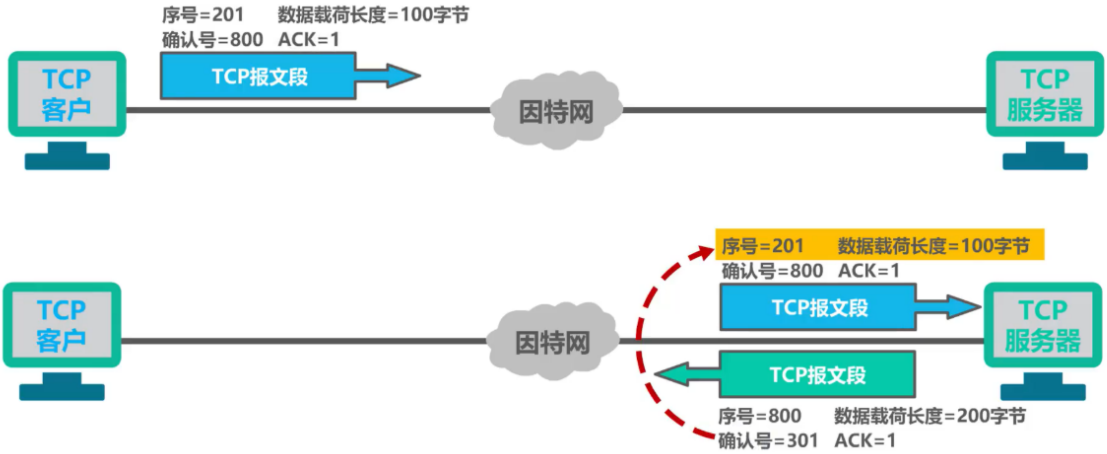
确认号：占32比特，取值范围[0, $2^{32}-1$]，确认号增加到最后一个后，下一个确认号就又回到0。

指出期望收到对方下一个TCP报文段的数据载荷的第一个字节的序号，同时也是对之前收到的所有数据的确认。

若确认号=n，则表明到序号n-1为止的所有数据都已正确接收，期望接收序号为n的数据。

确认标志位ACK：取值为1时确认号字段才有效；取值为0时确认号字段无效。

TCP规定，在连接建立后所有传送的TCP报文段都必须把ACK置1。



数据偏移、保留、窗口和校验和

数据偏移：占4比特，并以4字节为单位。

用来指出TCP报文段的数据载荷部分的起始处距离TCP报文段的起始处有多远。

这个字段实际上是指出了TCP报文段的首部长度。

首部固定长度为20字节，因此数据偏移字段的最小值为$(0101)_2$

首部最大长度为60字节，因此数据偏移字段的最大值为$(1111)_2$

保留：占6比特，保留为今后使用，但目前应置为0。

窗口：占16比特，以字节为单位。指出发送本报文段的一方的接收窗口。

窗口值作为接收方让发送方设置其发送窗口的依据。

这是以接收方的接收能力来控制发送方的发送能力，称为流量控制。

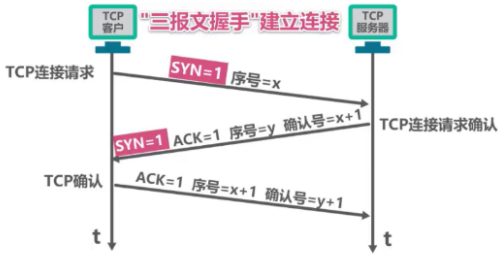
发送窗口的大小还取决于拥塞窗口的大小。

校验和：占16比特，检查范围包括TCP报文段的首部和数据载荷两部分。

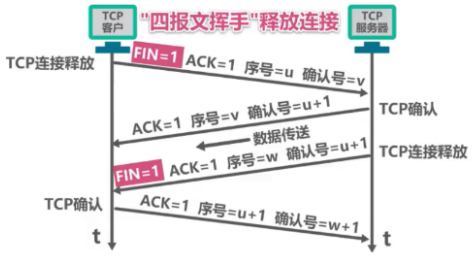
在计算校验和时，要在TCP报文段的前面加上12字节的伪首部。

同步标志位、终止标志位、复位标志位、推送标志位、紧急标志位和紧急指针

同步标志位SYN：在TCP连接建立时用来同步序号。



终止标志位FIN：用来释放TCP连接。



复位标志位RST：用来复位TCP连接。

当RST=1时，表明TCP连接出现了异常，必须释放连接，然后再重新建立连接。

RST置1还用来拒绝一个非法的报文段或拒绝打开一个TCP连接。

推送标志位PSH：接收方的TCP收到该标志位为1的报文段会**尽快上交应用进程**，而不必等到接收缓存都填满后再向上交付。

紧急标志位URG：取值为1时紧急指针字段有效；取值为0时紧急指针字段无效。

紧急指针：占16比特，以字节为单位，用来指明紧急数据的长度。

当发送方有紧急数据时，可将紧急数据插队到发送缓存的最前面，并立刻封装到一个TCP报文段中进行发送。紧急指针会指出本报文段数据载荷部分包含了多长的紧急数据，紧急数据之后是普通数据。

选项和填充

- **最大报文段长度MSS选项**：TCP报文段数据载荷部分的最大长度。
- **窗口扩大选项**：为了扩大窗口（提高吞吐率）。
- **时间戳选项**：
  - 用来计算往返时间RTT
  - 用于处理序号超范围的情况，又称为防止序号绕回PAWS。
- **选择确认选项**

填充：由于选项的长度可变，因此使用填充来**确保报文段首部能被4整除**（因为数据偏移字段，也就是首部长度字段，是以4字节为单位的）。